

Manual de Sobrevivência no Codeforces POPado

Rating 800

Autor: Kaeljane Ferreira da Silva

Data: Abril de 2026

Conteúdo

1	Nome do Problema (A. Exemplo)	2	10	Objetivos da Vitória (A. Goals of Victory)	19
2	Caixas de Halloumi (A. Halloumi Boxes)	3	11	Prática de Tiro (C. Target Practice)	21
3	Viagem em Linha (A. Line Trip)	5	12	Criança Ambiciosa (A. Ambitious Kid)	23
4	Coberto de Água (A. Cover in Water)	7	13	Jogo de Sequência (B. Sequence Game)	24
5	Jogo com Inteiros (A. Game with Integers)	9	14	Unidos Permaneceremos (A. United We Stand)	26
6	Trocas Irregulares (A. Jagged Swaps)	11	15	Botões (A. Buttons)	28
7	A Pintura da Doremy 3 (A. Doremy's Paint 3)	13	16	Colorindo o Array (A. Array Coloring)	30
8	Não Tente Contar (A. Don't Try to Count)	15	17	Desordenação (A. Desorting)	32
9	Quanto Custa Daytona? (A. How Much Does Daytona Cost?)	17	18	Inteiro Proibido (A. Forbidden Integer)	34
			19	Gafanhoto em uma Linha (A. Grasshopper on a Line)	36
			20	Array Unitário (A. Unit Array)	37
			21	Permutações Gêmeas (A. Twin Permutations)	39
			22	Espaço em Branco (B. Blank Space)	41
			23	Moedas (A. Coins)	43
			24	Mestre da Caminhada (A. Walking Master)	44

25	Precisamos do Zero (A. We Need the Zero)	46
26	Adicionar no Início e no Fim (C. Prepend and Append)	48
27	O Array de Serval e Mocha (A. Serval and Mocha's Array)	50
28	Um e Dois (A. One and Two)	52
29	Torne-o Bonito (A. Make it Beautiful)	54
30	Todo Mundo Gosta de Arrays Bons (A. Everybody Likes Good Arrays!)	56
31	Extremamente Redondo (A. Extremely Round)	57
32	Duas Permutações (A. Two Permutations)	59

1 Nome do Problema (A. Exemplo)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

(Aqui entrará a tradução do problema que nós fizemos).

Entrada e Saída

Entrada: (Regras da entrada...)

Saída: (Regras da saída...)

Exemplo

Input

3
1 2 3

Output

YES

A Sacada do Problema

(Aqui entrará a explicação detalhada do raciocínio lógico e da matemática por trás da solução).

Mental Chunking

- **Gatilho Visual:** (O que no enunciado ou restrição ($N \leq 10^5$) entrega o problema?)
- **Padrão Clássico:** (É uma Força Bruta? Guloso? Matemática? Array de Frequência?)
- **Armadilha Comum:** (Onde todo mundo toma Wrong Answer nesse tipo de questão?)

Solução (C++)

```
1 // O c digo corrigido e otimizado entrar aqui
2 void solve() {
3     // I can do it!
4 }
```

2 Caixas de Halloumi (A. Halloumi Boxes)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Theofanis está ocupado após sua última competição, pois agora ele tem que entregar muitos queijos halloumi por todo o mundo. Ele os armazenou dentro de n caixas e cada uma delas tem algum número a_i escrito nela.

Ele quer ordená-las em ordem não-decrescente (crescente) com base no número delas, no entanto, sua máquina funciona de uma maneira estranha. Ela só pode reverter qualquer subarray de caixas com comprimento **de no máximo** k .

Descubra se é possível ordenar as caixas usando **qualquer número de reversões**.

Reverter um subarray significa escolher dois índices i e j (onde $1 \leq i \leq j \leq n$) e mudar o array $a_1, a_2, \dots, a_n$ para $a_1, a_2, \dots, a_{i-1}, a_j, a_{j-1}, \dots, a_i, a_{j+1}, \dots, a_n$. O comprimento do subarray é então $j - i + 1$.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 100$) — o número de casos de teste. Cada caso de teste consiste em duas linhas. A primeira linha contém dois inteiros n e k ($1 \leq k \leq n \leq 100$) — o número de caixas e o comprimento máximo da reversão que Theofanis pode fazer. A segunda linha contém n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — o número escrito em cada caixa.

Saída: Para cada caso de teste, imprima YES se o array puder ser ordenado, ou NO caso contrário.

Exemplo

Input
5
3 2
1 2 3
3 1
9 9 9
4 4
6 4 2 1
4 3
10 3 830 14
2 1
3 1

Output
YES
YES
YES
YES
NO

A Sacada do Problema

O problema diz que podemos reverter qualquer pedaço do array que tenha tamanho **até** k . A palavra mágica aqui é "até"(ou *at most*). Isso significa que se $k = 3$, nós podemos escolher reverter pedaços de tamanho 3, tamanho 2 ou tamanho 1.

O que acontece se revertermos um subarray de **tamanho 2**? Nós simplesmente trocamos dois elementos vizinhos de lugar! Na Ciência da Computação, existe um algoritmo de ordenação chamado *Bubble Sort* que prova matematicamente que: **se você tem a capacidade de trocar qualquer par de elementos adjacentes infinitas vezes, você consegue ordenar abso-**

lutamente qualquer array.

Portanto, a lógica se resume a uma única pergunta:

- Se $k \geq 2$: Nós sempre podemos fazer operações de tamanho 2 (trocar vizinhos). Logo, a resposta é **sempre YES**.
- Se $k = 1$: Reverter um pedaço de tamanho 1 não faz nada (o elemento fica no mesmo lugar). Então, não conseguimos alterar o array. Nesse caso, a resposta só será **YES** se o array já vier ordenado de fábrica. Se vier bagunçado, é **NO**.

Mental Chunking

- **Gatilho Visual:** O problema pergunta "é possível ordenar?" permitindo uma operação em "subarrays de tamanho K ".
- **Padrão Clássico:** Redução ao *Bubble Sort*. Sempre que um problema permitir *swaps* (trocas) ou reversões de tamanho 2 ou maior de forma ilimitada, o array **sempre** poderá ser ordenado.
- **Armadilha Comum:** Ficar tentando simular as reversões usando laços complexos de repetição em vez de apenas verificar se $k \geq 2$.

Solução (C++)

```
1 void solve() {
2     int n, k;
3     cin >> n >> k;
4
5     vector<int> a(n);
6     for (int i = 0; i < n; i++) {
7         cin >> a[i];
8     }
9
10    // Se k for 2 ou maior, podemos fazer o Bubble Sort
11    if (k >= 2) {
12        cout << "YES\n";
13    }
```

```

14 // Se k for 1, a gente nao consegue mexer no array
15 else {
16     // Verifica se ja esta ordenado nativamente
17     if (is_sorted(all(a))) {
18         cout << "YES\n";
19     } else {
20         cout << "NO\n";
21     }
22 }
23 }

```

3 Viagem em Linha (A. Line Trip)

Tempo limite: 2 segundos

Memória limite: 256 megabytes

Enunciado

Existe uma estrada, que pode ser representada como uma reta numérica. Você está localizado no ponto 0 da reta, e deseja viajar do ponto 0 até o ponto x , e voltar para o ponto 0.

Você viaja de carro, que gasta 1 litro de gasolina por 1 unidade de distância percorrida. Quando você começa no ponto 0, seu carro está com o tanque cheio (seu tanque de gasolina contém a quantidade máxima possível de combustível).

Existem n postos de gasolina, localizados nos pontos $a_1, a_2, \dots, a_n$. Quando você chega a um posto de gasolina, você enche completamente o tanque do seu carro. **Note que você só pode reabastecer em postos de gasolina, e não há postos de gasolina nos pontos 0 e x .**

Você precisa calcular o volume mínimo possível do tanque de gasolina do seu carro (em litros) que permitirá que você viaje do ponto 0 até o ponto x e volte para o 0.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 1000$) — o número de casos de teste. A primeira linha de cada caso contém dois inteiros n e x ($1 \leq n \leq 50$; $2 \leq x \leq 100$). A segunda linha contém n inteiros $a_1, a_2, \dots, a_n$ ($0 < a_1 < a_2 < \dots < a_n < x$).

Saída: Para cada caso, imprima um único inteiro — a capacidade mínima do tanque.

Exemplo

Input
3
3 7
1 2 5
3 6
1 2 5
1 10
7

Output
4
3
7

A Sacada do Problema

O problema quer saber o tamanho do tanque. O tamanho do tanque define a distância máxima que você consegue percorrer **sem reabastecer**. Portanto, tudo o que precisamos encontrar é qual é a **maior distância entre dois postos consecutivos**.

A viagem tem duas partes: ir e voltar. Na ida, os espaços entre as paradas são:

- Do início até o 1º posto: $a_1 - 0$
- Entre os postos no meio do caminho: $a_i - a_{i-1}$

O grande segredo está no final da viagem: como não há posto de gasolina no destino final x , você precisa sair do último posto (a_n), chegar no destino final (x) e **voltar** para o último posto (a_n) usando apenas a gasolina que você abasteceu lá.

Isso significa que a distância que você percorre sem ajuda no final é o dobro da distância do último posto até o final: $2 \times (x - a_n)$. A resposta será simplesmente o valor máximo entre todas essas distâncias!

Mental Chunking

- **Gatilho Visual:** Reta numérica (1D), paradas em pontos específicos, viagem de ida e volta, busca por "capacidade mínima".
- **Padrão Clássico:** Distância Máxima entre Pontos (Greedy). O limite do sistema é sempre definido pelo maior gargalo (maior distância entre dois pontos consecutivos).
- **Armadilha Comum:** Esquecer que o carro precisa voltar do destino x até o posto a_n . A galera costuma colocar só $(x - a_n)$ no código e toma Wrong Answer. O correto é $2 \times (x - a_n)$.

Solução (C++)

```
1 void solve() {
2     int n, x;
3     cin >> n >> x;
4
5     vector<int> a(n);
6     for (int i = 0; i < n; i++) {
7         cin >> a[i];
8     }
9
10    // Distancia inicial (do zero ate o primeiro posto)
11    int ans = a[0];
12
13    // Distancias entre os postos intermediarios
14    for (int i = 1; i < n; i++) {
15        ans = max(ans, a[i] - a[i-1]);
16    }
17
18    // O gargalo final (ir do ultimo posto ate o fim e VOLTAR)
19    ans = max(ans, 2 * (x - a[n-1]));
20
21    cout << ans << '\n';
22 }
```

4 Coberto de Água (A. Cover in Water)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Filip tem uma fileira de células, algumas das quais estão bloqueadas e outras estão vazias. Ele quer que todas as células vazias tenham água. Ele tem duas ações à sua disposição:

- **1** — colocar água em uma célula vazia.
- **2** — remover água de uma célula e colocá-la em qualquer outra célula vazia.

Se em algum momento a célula i ($2 \leq i \leq n-1$) estiver vazia e ambas as células $i-1$ e $i+1$ contiverem água, então ela se enche de água automaticamente.

Encontre o número mínimo de vezes que ele precisa realizar a **ação 1** para encher todas as células vazias com água.

Note que você não precisa minimizar o uso da ação 2. Note que células bloqueadas não contêm água nem Filip pode colocar água nelas.

Entrada e Saída

Entrada: A primeira linha contém o número de casos de teste t ($1 \leq t \leq 100$). A primeira linha de cada caso contém um único inteiro n ($1 \leq n \leq 100$) — o número de células. A próxima linha contém uma string s de comprimento n . O i -ésimo caractere de s é '.' se a célula i estiver vazia e '#' se a célula i estiver bloqueada.

Saída: Para cada caso de teste, imprima um único número — a quantidade mínima de ações 1 necessárias para encher todas as células vazias com água.

Exemplo

Input

```
5
3
...
7
##...#
7
..#.#..
4
####
10
#...#...#
```

Output

```
2
2
5
0
2
```

A Sacada do Problema

O segredo deste problema está em ler as entrelinhas: **a ação 2 é de graça!** Nós só precisamos economizar a ação 1 (criar água do nada).

A mecânica automática diz que se tivermos o padrão **Água - Vazio - Água**, o meio enche sozinho. O que acontece se tivermos pelo menos 3 espaços vazios consecutivos (...)?

1. Usamos 2 ações do tipo 1 para colocar água nas pontas: A.A.
2. O meio enche automaticamente: AAA.

3. Agora vem a mágica: podemos usar a ação 2 (que é grátis) para tirar a água do meio e jogar em qualquer outro lugar do mapa.
4. O espaço fica A.A de novo... e adivinha? **Ele enche sozinho novamente!**

Acabamos de criar uma **fonte infinita de água**! Usando apenas 2 ações do tipo 1, podemos gerar água infinita para preencher todo o resto do array usando a ação 2.

Portanto, a lógica é binária:

- Se existir **qualquer** buraco com 3 ou mais espaços vazios consecutivos, a resposta é **sempre 2**.
- Se não existir nenhum espaço de tamanho 3 (o máximo é 1 ou 2), não conseguimos criar a fonte infinita. Logo, somos obrigados a colocar água manualmente em cada espaço vazio. A resposta será simplesmente o **total de pontos (.)** na string.

Mental Chunking

- **Gatilho Visual:** Uma operação custa pontos (Ação 1) e outra é de graça (Ação 2), com uma regra de propagação baseada em vizinhos.
- **Padrão Clássico:** Ad-hoc / Busca por Padrão de Fonte Infinita. Em problemas de "espalhamento", procure sempre pela menor estrutura capaz de se auto-sustentar ou gerar recursos infinitos.
- **Armadilha Comum:** Tentar simular cada passo (colocar água, mover, checar) usando laços confusos. A solução requer apenas analisar a string estaticamente.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     string s;
5     cin >> s;
6
7     int total_vazios = 0;
8     int max_consecutivos = 0;
9     int atual_consecutivos = 0;
10
11     for (int i = 0; i < n; i++) {
12         if (s[i] == ',') {
13             total_vazios++;
14             atual_consecutivos++;
15             max_consecutivos = max(max_consecutivos,
16                                     atual_consecutivos);
17         } else { // Achou um '#'
18             atual_consecutivos = 0; // Zera o contador de
19                                     sequencia
20         }
21     }
22
23     // Se achamos uma fonte infinita (3 ou mais pontos seguidos)
24     if (max_consecutivos >= 3) {
25         cout << 2 << '\n';
26     } else {
27         // Se nao, temos que encher cada buraco manualmente
28         cout << total_vazios << '\n';
29     }
30 }
```


5 Jogo com Inteiros (A. Game with Integers)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Vanya e Vova estão jogando um jogo. Os jogadores recebem um inteiro n . No seu turno, o jogador pode somar 1 ao inteiro atual ou subtrair 1. Os jogadores se alternam; Vanya começa. Se **após** a jogada de Vanya o inteiro for divisível por 3, então ele vence. Se 10 jogadas se passarem e Vanya não tiver vencido, então Vova vence.

Escreva um programa que, com base no inteiro n , determine quem vencerá se ambos os jogadores jogarem de forma ideal.

Entrada e Saída

Entrada: A primeira linha contém o inteiro t ($1 \leq t \leq 100$) — o número de casos de teste. A linha única de cada caso de teste contém o inteiro n ($1 \leq n \leq 1000$).

Saída: Para cada caso de teste, imprima **First** sem aspas se Vanya vencer, e **Second** sem aspas se Vova vencer.

Exemplo

Input

```
6
1
3
5
100
999
1000
```

Output

```
First
Second
First
First
Second
First
```

A Sacada do Problema

Na Teoria dos Jogos, quando o problema diz "ambos jogam de forma ideal", não precisamos simular uma árvore complexa de decisões, basta analisar as opções do primeiro jogador (Vanya).

Vanya quer que, após o turno **dele**, o número seja um múltiplo de 3. Ele pode fazer +1 ou -1.

- Se n deixar resto 1 na divisão por 3 (ex: 4, 7, 10), Vanya subtrai 1 e ganha **imediatamente no turno 1**.
- Se n deixar resto 2 na divisão por 3 (ex: 5, 8, 11), Vanya soma 1 e ganha **imediatamente no turno 1**.

O único cenário onde Vanya não ganha de cara é se n **já for múltiplo de 3** desde o início (ex: 3, 6, 9). Nesse caso, qualquer jogada de Vanya (somar ou subtrair 1) vai "estragar" o número, transformando-o num não-múltiplo de 3. Aí entra a jogada ideal do Vova: Vova quer que o jogo dure até acabar os 10 turnos. Para fazer isso, ele só precisa **desfazer** a jogada do Vanya! Se o Vanya somar 1, o Vova subtrai 1.

Isso cria um loop infinito perfeito: Vanya estraga o múltiplo, Vova conserta. Como Vanya só ganha se o número for múltiplo de 3 **após a jogada dele**, e ele é sempre forçado a sair do múltiplo, ele nunca vai ganhar.

Portanto, a lógica é ridícula de simples: Se n é múltiplo de 3, Vova ganha (**Second**). Se não for, Vanya ganha (**First**). Aquela regra de "10 jogadas" é só uma distração (uma armadilha de complexidade)!

Mental Chunking

- **Gatilho Visual:** O problema descreve um jogo de turnos entre duas pessoas (Vanya/Vova, Alice/Bob) onde se pode fazer $+1/-1$ e pede quem vence se "ambos jogam de forma ideal".
- **Padrão Clássico:** Teoria dos Jogos (Estratégia de Espelhamento). Se o jogador 1 não consegue vencer no primeiro turno, veja se o jogador 2 consegue "espelhar" ou "desfazer" a jogada infinitamente para garantir o empate/vitória por tempo.
- **Armadilha Comum:** Tentar simular o jogo com laços **for** rodando 10 vezes ou usar algoritmos recursivos como Minimax. A resposta é quase sempre **apenas um if(modulo)**.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4
5     // Se for multiplo de 3, a jogada ideal de Vova garante a
6     // vitoria dele
7     if (n % 3 == 0) {
```

```
7         cout << "Second\n";
8     }
9     // Caso contrario, Vanya consegue forçar um multiplo de 3 de
10    imediato
11    else {
12        cout << "First\n";
13    }
```

6 Trocas Irregulares (A. Jagged Swaps)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Você recebe uma permutação a de tamanho n . Você pode realizar a seguinte operação:

- Selecione um índice i de 2 a $n - 1$ tal que $a_{i-1} < a_i$ e $a_i > a_{i+1}$. Troque a_i e a_{i+1} de lugar.

Determine se é possível ordenar a permutação após um número finito de operações.

Uma permutação é um array que consiste em n inteiros distintos de 1 a n em ordem arbitrária. Por exemplo, $[2, 3, 1, 5, 4]$ é uma permutação, mas $[1, 2, 2]$ não é uma permutação (2 aparece duas vezes no array) e $[1, 3, 4]$ também não é uma permutação ($n = 3$, mas há um 4 no array).

Entrada e Saída

Entrada: Cada teste contém múltiplos casos de teste. A primeira linha contém o número de casos de teste t ($1 \leq t \leq 5000$). A primeira linha de cada caso de teste contém um único inteiro n ($3 \leq n \leq 10$) — o tamanho da permutação. A segunda linha de cada caso de teste contém n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$) — os elementos da permutação a .

Saída: Para cada caso de teste, imprima YES se for possível ordenar a permutação, e NO caso contrário.

Exemplo

Input

```
6
3
1 2 3
5
1 3 2 5 4
5
5 4 3 2 1
3
3 1 2
4
2 3 1 4
5
5 1 2 3 4
```

Output

```
YES
YES
NO
NO
NO
NO
```

A Sacada do Problema

O problema nos permite escolher um índice i e trocar o elemento na posição i com o elemento na posição $i + 1$. No entanto, olhe atentamente para a restrição fundamental da operação: o índice escolhido deve estar entre **2 e $n-1$** .

O que isso significa na prática? Significa que nós **nunca podemos escolher** $i = 1$. Como a única forma de mover o primeiríssimo elemento do array (a_1)

seria escolhendo $i = 1$ para trocá-lo com a_2 , chegamos à conclusão de que **o primeiro elemento do array está congelado para sempre**. Ele não pode ser movido para a direita, e ninguém pode ser movido para a esquerda dele.

Como o objetivo é ordenar uma permutação de 1 a n , o array ordenado obrigatoriamente deve começar com o número 1. Se o nosso array começar com qualquer número diferente de 1, é fisicamente impossível ordená-lo, pois esse número intruso nunca sairá da primeira posição. A resposta será NO.

E se o array começar com 1? Como 1 é o menor número possível, qualquer elemento à direita dele será maior, o que sempre satisfará a condição $a_{i-1} < a_i$ para podermos empurrar os números grandes para a direita (um comportamento quase idêntico ao *Bubble Sort*). Portanto, se $a_1 = 1$, a resposta é sempre YES.

Mental Chunking

- **Gatilho Visual:** Operações de troca em arrays onde as extremidades ($i = 1$ ou $i = n$) são excluídas das opções de escolha.
- **Padrão Clássico:** Invariantes / Elementos Fixos (Ad-hoc). Se as regras do jogo não permitem tocar em uma peça específica, verifique imediatamente se essa peça já nasceu na posição de vitória.
- **Armadilha Comum:** Olhar para o limite minúsculo ($n \leq 10$) e achar que é necessário codar uma simulação de força bruta ou BFS para testar todas as trocas possíveis, gastando dezenas de linhas de código à toa.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4
5     vector<int> a(n);
6     for (int i = 0; i < n; i++) {
```

```
7         cin >> a[i];
8     }
9
10    // Se o primeiro elemento for 1, a resposta e YES.
11    // Caso contrario, e NO, pois ele nunca podera ser movido.
12    if (a[0] == 1) {
13        cout << "YES\n";
14    } else {
15        cout << "NO\n";
16    }
17 }
```

7 A Pintura da Doremy 3 (A. Doremy's Paint 3)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Um array $b_1, b_2, \dots, b_n$ de inteiros positivos é considerado **bom** se todas as somas de dois elementos adjacentes forem iguais ao mesmo valor. Mais formalmente, o array é bom se existir um k tal que $b_1 + b_2 = b_2 + b_3 = \dots = b_{n-1} + b_n = k$.

Doremy tem um array a de comprimento n . Agora Doremy pode permutar seus elementos (mudar a ordem deles) como ela quiser. Determine se ela consegue tornar o array bom.

Entrada e Saída

Entrada: A entrada consiste em múltiplos casos de teste. A primeira linha contém um único inteiro t ($1 \leq t \leq 100$) — o número de casos de teste. A primeira linha de cada caso de teste contém um único inteiro n ($2 \leq n \leq 100$) — o comprimento do array a . A segunda linha contém n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^5$).

Saída: Para cada caso de teste, imprima **Yes** se for possível tornar o array bom, e **No** caso contrário. Você pode imprimir a resposta em qualquer caso (maiúsculas ou minúsculas).

Exemplo

Input

```
5
2
8 9
3
1 1 2
4
1 1 4 5
5
2 3 3 3 3
4
100000 100000 100000 100000
```

Output

```
Yes
Yes
No
No
Yes
```

A Sacada do Problema

O problema exige que a soma de qualquer par de adjacentes seja igual a k . Vamos escrever isso matematicamente pegando os três primeiros elementos: $b_1 + b_2 = b_2 + b_3$

Se você subtrair b_2 dos dois lados da equação, o que sobra? $b_1 = b_3$

Se fizermos isso para os próximos elementos ($b_2 + b_3 = b_3 + b_4$), descobrimos que $b_2 = b_4$. Isso significa que para o array ser "bom", ele **obrigatoriamente** precisa seguir um padrão alternado: $x, y, x, y, x, y, \dots$

Sabendo disso, tiramos três conclusões de ouro:

1. Se o array tiver **3 ou mais números diferentes**, é fisicamente impossível formar um padrão alternado de duas posições. Resposta: **No**.
2. Se o array tiver **apenas 1 número** (todos os elementos iguais), a soma dos vizinhos sempre será a mesma. Resposta: **Yes**.
3. Se o array tiver **exatamente 2 números diferentes**, precisamos checar se a quantidade deles é equilibrada. Para podermos intercalar x e y perfeitamente, a diferença entre a frequência (quantidade) do primeiro número e do segundo número não pode ser maior que 1. (ex: para $N = 5$, podemos ter três X e dois Y , formando X, Y, X, Y, X). Se a diferença for ≤ 1 , **Yes**, senão **No**.

Para resolver, basta usar um 'std::map' para contar a frequência de cada número e aplicar essas três regrinhas lógicas!

Mental Chunking

- **Gatilho Visual:** O problema pede para reordenar um array para que equações de vizinhos ($A_i + A_{i+1} = K$) sejam satisfeitas.
- **Padrão Clássico:** Redução Algébrica + Mapa de Frequência. Sempre que houver uma equação de igualdade com termos em comum ($a + b = b + c$), corte o termo do meio para descobrir a relação real ($a = c$).
- **Armadilha Comum:** Tentar ordenar o array e intercalar manualmente (com *two pointers*), o que gera um código enorme e propenso a bugs. Apenas contar as frequências é suficiente.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4
5     // O map vai guardar: chave = numero, valor = quantas vezes
6     // ele aparece
7     map<int, int> freq;
```

```
7     for (int i = 0; i < n; i++) {
8         int x;
9         cin >> x;
10        freq[x]++;
11    }
12
13    // Regra 1: Mais de 2 numeros distintos? Impossivel alternar.
14    if (freq.size() >= 3) {
15        cout << "No\n";
16    }
17    // Regra 2: Apenas 1 numero distinto? Sempre da certo.
18    else if (freq.size() == 1) {
19        cout << "Yes\n";
20    }
21    // Regra 3: Exatamente 2 numeros distintos.
22    else {
23        auto it = freq.begin();
24        int qtd1 = it->second; // Frequencia do primeiro numero
25        it++;
26        int qtd2 = it->second; // Frequencia do segundo numero
27
28        // A diferenca de quantidade entre eles pode ser no
29        // maximo 1
30        if (abs(qtd1 - qtd2) <= 1) {
31            cout << "Yes\n";
32        } else {
33            cout << "No\n";
34        }
35    }
36 }
```


Output

```
3
1
2
-1
1
0
1
3
-1
0
2
5
```

A Sacada do Problema

Sempre que uma operação pede para você "concatenar uma coisa com ela mesma", o que está acontecendo na verdade é um **crescimento exponencial**. O tamanho da string dobra a cada passo ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \dots$).

O problema nos diz que $n \cdot m \leq 25$, o que na prática significa que a string s procurada tem um tamanho máximo de **25 caracteres**. Se o tamanho máximo procurado é 25, a nossa string x só precisa crescer até ficar com o tamanho de s ou, no máximo, o dobro disso (para garantir que a palavra não quebre no meio da junção de duas strings antigas).

Pense comigo: mesmo no pior caso absoluto, onde a string inicial x tem tamanho 1, se nós fizermos a operação **6 vezes**, o tamanho final de x será $2^6 = 64$. O tamanho 64 é mais do que suficiente para engolir qualquer string s de tamanho 25.

Portanto, não precisamos criar fórmulas malucas para checar os padrões. Nós podemos, literalmente, usar um 'while', simular as concatenações e usar a função '.find()' nativa do C++! Se fizermos a operação até umas 6 ou 7 vezes e a string s ainda não tiver aparecido dentro do x , é garantido que ela **nunca** vai aparecer.

Mental Chunking

- **Gatilho Visual:** Operação de auto-concatenação ($x = x + x$) aliada a restrições de limite extremamente pequenas na entrada ($N \cdot M \leq 25$).
- **Padrão Clássico:** Simulação Exponencial Segura. Quando algo dobra a cada iteração, ele rompe qualquer limite em pouquíssimos passos (normalmente < 10). É o sinal verde para codar a Força Bruta direta.
- **Armadilha Comum:** Tentar fazer matemática para prever exatamente em qual quebra de limite as strings se encontram, resultando em códigos complexos para resolver casos base (x contido em s , vs s contido em x). O loop 'while(ops <= 6)' é imbatível.

Solução (C++)

```
1 void solve() {
2     int n, m;
3     cin >> n >> m;
4
5     string x, s;
6     cin >> x >> s;
7
8     int ops = 0;
9
10    // 6 operacoes transformam uma string de tamanho 1
11    // em uma string de tamanho 64. Mais do que suficiente!
12    while (ops <= 6) {
13        // Se a string s for encontrada dentro de x
14        // string::npos significa "not found"
15        if (x.find(s) != string::npos) {
16            cout << ops << '\n';
17            return;
18        }
19
20        // Se nao achou, dobra o tamanho do x e tenta de novo
21        x += x;
22        ops++;
23    }
24 }
```



```
25 // Se o loop terminou e nao retornou, e impossivel
26 cout << -1 << '\n';
27 }
```

9 Quanto Custa Daytona? (A. How Much Does Daytona Cost?)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Definimos um número inteiro como sendo o **mais comum** em um subsegmento se o seu número de ocorrências naquele subsegmento for maior do que o número de ocorrências de qualquer outro inteiro naquele subsegmento. Um subsegmento de um array é um segmento contíguo de elementos no array a .

Dado um array a de tamanho n e um inteiro k , determine se existe um subsegmento não vazio de a onde k seja o elemento *mais comum*.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 1000$) — o número de casos de teste. A primeira linha de cada caso contém dois inteiros n e k ($1 \leq n \leq 100, 1 \leq k \leq 100$) — o número de elementos no array e o elemento que deve ser o mais comum. A segunda linha de cada caso contém n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 100$).

Saída: Para cada caso de teste, imprima **YES** se existir um subsegmento no qual k seja o elemento mais comum, e **NO** caso contrário.

Exemplo

Input
7
5 4
1 4 3 4 1
4 1
2 3 4 4
5 6
43 5 60 4 2
2 5
1 5
4 1
5 3 3 1
1 3
3
5 3
3 4 1 5 5

Output
YES
NO
NO
YES
YES
YES
YES

A Sacada do Problema

O problema pergunta se existe **qualquer** subsegmento onde k é o mais frequente. O erro comum aqui é tentar olhar para o array inteiro ou tentar calcular janelas deslizantes.

Pense no menor subsegmento possível: um segmento de tamanho 1. Se pegarmos um subsegmento que contém apenas um elemento, e esse elemento for o próprio k (ou seja, $[k]$), quantas vezes o k aparece? **Uma vez**. Quantas vezes qualquer outro número aparece? **Zero vezes**.

Portanto, em um subsegmento de tamanho 1 contendo k , o k é, por definição, o elemento mais comum.

A conclusão lógica é: se o número k aparece em **qualquer lugar** do array original, nós podemos simplesmente escolher o subsegmento que contém apenas ele, e a condição será satisfeita. Se k não estiver no array, nenhum subsegmento poderá tê-lo como o mais comum.

Mental Chunking

- **Gatilho Visual:** O problema pede a existência de um subsegmento com uma propriedade de "máximo" ou "mais comum".
- **Padrão Clássico:** Subsegmento Unitário (Mínimo). Se a propriedade pode ser satisfeita por um único elemento, não há necessidade de verificar segmentos maiores.
- **Armadilha Comum:** Tentar contar a frequência de k no array inteiro. Mesmo que outro número apareça mais vezes no total, o problema pede apenas a existência de *algum* subsegmento.

Solução (C++)

```
1 void solve() {  
2     int n, k;  
3     cin >> n >> k;  
4  
5     bool encontrou = false;  
6     for (int i = 0; i < n; i++) {  
7         int x;  
8         cin >> x;  
9         if (x == k) {  
10             encontrou = true;  
11         }  
12     }
```

```

13
14 // Se o k existir no array, podemos isolar ele
15 // em um subsegmento de tamanho 1.
16 if (encontrou) {
17     cout << "YES\n";
18 } else {
19     cout << "NO\n";
20 }
21 }

```

10 Objetivos da Vitória (A. Goals of Victory)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Existem n times em um torneio de futebol. Cada par de times se enfrenta exatamente uma vez. Após cada partida, Pak Chanek recebe dois inteiros como resultado: o número de gols que os dois times marcaram durante a partida.

A **eficiência** de um time é igual ao número total de gols que o time marcou em todas as suas partidas menos o número total de gols marcados pelos adversários em todas as suas partidas.

Após o término do torneio, Pak Dengklek conta a eficiência de cada time. Acontece que ele esqueceu a eficiência de um dos times. Dada a eficiência de $n - 1$ times $a_1, a_2, a_3, \dots, a_{n-1}$, qual é a eficiência do time que falta? Pode-se mostrar que a eficiência do time ausente pode ser determinada de forma única.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 500$) — o número de casos de teste. A primeira linha de cada caso contém um inteiro n ($2 \leq n \leq 100$) — o número de times. A segunda linha contém $n - 1$ inteiros $a_1, a_2, \dots, a_{n-1}$ ($-100 \leq a_i \leq 100$) — a eficiência dos $n - 1$ times.

Saída: Para cada caso de teste, imprima um inteiro representando a eficiência do time que falta.

Exemplo

Input
2
4
3 -4 5
11
-30 12 -57 7 0 -81 -68 41 -89 0

Output
-4
265

A Sacada do Problema

A chave para resolver este problema é entender que o sistema de "eficiência" em um torneio fechado é um sistema de **Soma Zero**.

Pense em um único gol marcado no torneio. Quando o Time A marca um gol contra o Time B:

- A eficiência do Time A aumenta em 1 (gol marcado).
- A eficiência do Time B diminui em 1 (gol sofrido).

A soma das eficiências desses dois times após esse gol é: $+1 + (-1) = 0$.

Como cada gol marcado por qualquer time no torneio é, obrigatoriamente, um gol sofrido por outro time, se somarmos a eficiência de **todos** os times participantes, o resultado deve ser sempre **0**.

Portanto, se temos $n - 1$ eficiências e falta uma (X), temos a equação:

$$\sum_{i=1}^{n-1} a_i + X = 0 \implies X = -\sum_{i=1}^{n-1} a_i$$

A solução é simplesmente somar todas as eficiências dadas e inverter o sinal do resultado final.

Mental Chunking

- **Gatilho Visual:** Estatísticas de saldo ou eficiência em um torneio onde todos jogam contra todos e falta a informação de um participante.
- **Padrão Clássico:** Invariante de Soma Zero. Em qualquer sistema onde o ganho de um é a perda de outro, a soma total das variações é constante (geralmente zero).
- **Armadilha Comum:** Tentar usar o número de times n para calcular quantos jogos cada time jogou ($n - 1$) ou tentar simular placares individuais. A eficiência total não depende do número de jogos.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4
5     int soma_eficiencia = 0;
6     // 0 problema nos da n-1 valores
7     for (int i = 0; i < n - 1; i++) {
8         int a;
9         cin >> a;
10        soma_eficiencia += a;
11    }
12
13    // Como a soma total deve ser 0:
14    // soma_conhecida + X = 0 -> X = -soma_conhecida
15    cout << -soma_eficiencia << '\n';
16 }
```

11 Prática de Tiro (C. Target Practice)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Um alvo de 10×10 é feito de cinco "anéis", como mostrado na figura. Cada anel tem um valor de pontuação diferente: o anel mais externo vale 1 ponto, o próximo vale 2 pontos, ..., e o anel central vale 5 pontos.

Vlad disparou várias flechas no alvo. Ajude-o a determinar quantos pontos ele marcou.

Entrada e Saída

Entrada: A entrada consiste em múltiplos casos de teste. A primeira linha contém um único inteiro t ($1 \leq t \leq 1000$) — o número de casos de teste. Cada caso de teste consiste em 10 linhas, cada uma contendo 10 caracteres. Cada caractere na grade é ou 'X' (representando uma flecha) ou '.' (representando nenhuma flecha).

Saída: Para cada caso de teste, imprima um único inteiro — o número total de pontos das flechas.

Exemplo

Input

```
4
X.....
.....
.....X..
....X....
.....X..
.....
.....X
..X.....
.....
.....X
(restante do exemplo omitido por brevidade)
```

Output

```
17
0
5
220
```

A Sacada do Problema

O desafio aqui é identificar em qual "anel" uma coordenada (r, c) se encontra. Imagine que as linhas e colunas vão de 0 a 9.

Para qualquer célula, a sua "profundidade" em relação às bordas define o seu valor. Uma célula está a uma certa distância de quatro bordas:

- Distância do topo: r
- Distância da base: $9 - r$
- Distância da esquerda: c

- Distância da direita: $9 - c$

O anel ao qual a célula pertence é determinado pela **menor dessas distâncias**. Por exemplo, a célula (0,0) tem distâncias (0,9,0,9). O mínimo é 0. Como o primeiro anel vale 1 ponto, a fórmula é $\min(\text{distâncias}) + 1$. A célula (4,4) tem distâncias (4,5,4,5). O mínimo é 4, então ela vale $4 + 1 = 5$ pontos.

Mental Chunking

- **Gatilho Visual:** Grades com camadas concêntricas (anéis, molduras, cebolas).
- **Padrão Clássico:** Distância Mínima às Bordas. Em uma grade $N \times N$, o valor de uma camada é dado por $\min(r, (N - 1) - r, c, (N - 1) - c)$.
- **Armadilha Comum:** Tentar hardcodar os anéis com vários if ou criar uma matriz de pontos manualmente. Isso gasta tempo e gera bugs bobos de digitação.

Solução (C++)

```

1 void solve() {
2     int total = 0;
3     for (int i = 0; i < 10; i++) {
4         for (int j = 0; j < 10; j++) {
5             char c;
6             cin >> c;
7             if (c == 'X') {
8                 // Calcula a distancia minima ate qualquer uma
9                 das 4 bordas
10                int dist_r = min(i, 9 - i);
11                int dist_c = min(j, 9 - j);
12                int anel = min(dist_r, dist_c);
13
14                total += (anel + 1);
15            }
16        }
17    }
18    cout << total << '\n';
19 }
```

12 Criança Ambiciosa (A. Ambitious Kid)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Chaneka, filha de Pak Chanek, é uma criança ambiciosa, então Pak Chanek propõe o seguinte problema para testar sua ambição.

Dado um array de inteiros $[A_1, A_2, A_3, \dots, A_N]$. Em uma operação, Chaneka pode escolher um elemento e aumentar ou diminuir o valor desse elemento em 1. Ela pode realizar essa operação várias vezes, inclusive para diferentes elementos.

Qual é o número mínimo de operações que devem ser feitas para que $A_1 \times A_2 \times A_3 \times \dots \times A_N = 0$?

Entrada e Saída

Entrada: A primeira linha contém um único inteiro N ($1 \leq N \leq 10^5$). A segunda linha contém N inteiros $A_1, A_2, A_3, \dots, A_N$ ($-10^5 \leq A_i \leq 10^5$).

Saída: Imprima um inteiro representando o número mínimo de operações para que o produto do array seja igual a 0.

Exemplo

Input

```
3
2 -6 5
```

Output

```
2
```

A Sacada do Problema

Para que o produto de vários números seja igual a zero, existe uma regra matemática absoluta: **pelo menos um dos números deve ser zero**.

O problema nos permite transformar qualquer número do array em zero através de incrementos ou decrementos. O custo para transformar um número X em 0 é exatamente a sua distância até a origem na reta numérica, ou seja, o seu **valor absoluto** ($|X|$).

Como queremos o número *mínimo* de operações, não faz sentido tentar transformar mais de um número em zero. Basta escolhermos aquele que já está mais perto do zero desde o início. Portanto, a resposta é simplesmente o menor valor absoluto presente no array.

Mental Chunking

- **Gatilho Visual:** O problema pede para tornar o *produto* de um array igual a zero com o menor custo de alterações.
- **Padrão Clássico:** Propriedade do Elemento Neutro da Multiplicação. Para zerar um produto, você só precisa de um único multiplicador zero. O custo de zerar A_i é $\text{abs}(A_i)$.
- **Armadilha Comum:** Tentar calcular o produto real do array. Como os números podem chegar a 10^5 e o array tem 10^5 elementos, o produto seria um número com milhões de dígitos, causando *overflow* imediato.

Solução (C++)

```
1 void solve() {
2     int n;
```

```

3  cin >> n;
4
5  // Começamos com um valor infinito para achar o mínimo
6  int min_ops = 1e9;
7
8  for (int i = 0; i < n; i++) {
9      int a;
10     cin >> a;
11
12     // A distancia de qualquer numero ate o zero e seu valor
13     absoluto
14     int custo = abs(a);
15
16     if (custo < min_ops) {
17         min_ops = custo;
18     }
19
20     cout << min_ops << '\n';
21 }

```

13 Jogo de Sequência (B. Sequence Game)

Tempo limite: 2 segundos

Memória limite: 256 megabytes

Enunciado

Tema e Vika estão jogando o seguinte jogo:

Primeiro, Vika cria uma sequência de inteiros positivos a de comprimento m e a escreve em um papel. Depois, ela pega uma nova folha e escreve a sequência b seguindo estas regras:

- Primeiro, ela escreve a_1 .
- Depois, ela escreve apenas os a_i ($2 \leq i \leq m$) tais que $a_{i-1} \leq a_i$.

Por exemplo, da sequência $a = [4, 3, 2, 6, 3, 3]$, Vika obterá $b = [4, 6, 3]$. Ela dá a sequência b para Tema, que tenta adivinhar a sequência a . Ajude-o a encontrar **pelo menos uma** sequência a válida.

Nota: O comprimento da sua sequência de saída não deve exceder o dobro do comprimento da sequência de entrada ($m \leq 2n$).

Entrada e Saída

Entrada: A primeira linha contém t ($1 \leq t \leq 10^4$) — o número de casos de teste. Cada caso começa com n ($1 \leq n \leq 2 \cdot 10^5$) — o comprimento de b . A próxima linha contém os n inteiros $b_1, b_2, \dots, b_n$.

Saída: Para cada caso, imprima o comprimento m da sua sequência a e, na linha seguinte, os m inteiros que a compõem.

Exemplo

Input
6
3
4 6 3
3
1 2 3
5
1 7 9 5 7

Output
6
4 3 2 6 3 3
3
1 2 3
6
1 7 9 3 5 7

A Sacada do Problema

O problema nos dá a sequência "filtrada" (b) e quer que a gente recupere uma possível sequência original (a). A regra da Vika diz: "Eu só mantenho um número se ele for maior ou igual ao anterior".

Nossa estratégia será reconstruir a percorrendo b . Para cada elemento b_i (começando do segundo):

1. Se $b_i \geq b_{i-1}$, tudo certo! Podemos apenas colocar b_i na nossa sequência a , pois ele passaria no filtro naturalmente.
2. Se $b_i < b_{i-1}$, temos um problema. Se colocarmos b_i direto depois de b_{i-1} , ele seria descartado pelo filtro da Vika. Precisamos de uma "ponte".

A "ponte" consiste em inserir um número extra antes de b_i que seja pequeno o suficiente para que b_i seja aceito. Qual o número mais seguro? O próprio b_i ! Se inserirmos b_i antes de b_i , teremos a sequência $[\dots, b_{i-1}, b_i, b_i]$. Como $b_i \leq b_i$ é verdade, o segundo b_i será mantido no filtro e aparecerá na sequência b .

Como inserimos no máximo um número extra para cada número já existente, nossa sequência a terá no máximo tamanho $2n$, respeitando o limite do problema.

Mental Chunking

- **Gatilho Visual:** O problema pede para reconstruir uma sequência original dado um filtro de "não-decrescente" ($a_{i-1} \leq a_i$), permitindo que a saída seja maior que a entrada.
- **Padrão Clássico:** Construção com Elementos de Preenchimento (Bridge Elements). Quando uma condição de vizinhança falha, insira um elemento extra que satisfaça a condição para o próximo número.
- **Armadilha Comum:** Tentar remover elementos ou pensar em soluções complexas de busca. Em problemas de construção com limites folgados ($2n$), a solução mais simples (repetir elementos) costuma ser a correta.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     vector<int> b(n);
5     for(int i = 0; i < n; i++) cin >> b[i];
6
7     vector<int> a;
8     a.push_back(b[0]);
9
10    for(int i = 1; i < n; i++) {
11        // Se a regra for quebrada (o atual é menor que o
           anterior)
```

```

12     if (b[i] < b[i-1]) {
13         // Inserimos o proprio b[i] como "ponte"
14         a.push_back(b[i]);
15     }
16     a.push_back(b[i]);
17 }
18
19 cout << a.size() << '\n';
20 for(int x : a) cout << x << " ";
21 cout << '\n';
22 }

```

14 Unidos Permaneceremos (A. United We Stand)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Dado um array a de comprimento n , contendo inteiros. Existem dois arrays inicialmente vazios b e c . Você precisa adicionar cada elemento do array a a **exatamente um** dos arrays b ou c , de modo a satisfazer as seguintes condições:

- Ambos os arrays b e c devem ser não-vazios ($l_b, l_c \geq 1$).
- Para quaisquer dois índices i e j , c_j **não é** um divisor de b_i .

Imprima os arrays b e c que podem ser obtidos, ou imprima -1 se eles não existirem.

Entrada e Saída

Entrada: A primeira linha contém o inteiro t ($1 \leq t \leq 500$) — o número de casos de teste. A primeira linha de cada caso contém o inteiro n ($2 \leq n \leq 100$) — o comprimento de a . A segunda linha contém n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$).

Saída: Se não houver solução, imprima -1 . Caso contrário, na primeira linha imprima os comprimentos l_b e l_c . Na segunda linha, imprima os elementos de b . Na terceira linha, imprima os elementos de c .

Exemplo

Input

```
5
3
2 2 2
5
1 2 3 4 5
3
1 3 5
7
1 7 7 2 9 1 4
5
4 8 12 12 4
```

Output

```
-1
3 2
1 3 5
2 4
1 2
1
3 5
2 5
1 1 2 4 7 7
9
3 2
4 4 8
12 12
```

A Sacada do Problema

A condição principal é: nenhum elemento de c pode dividir qualquer elemento de b . Sabemos que, para números inteiros positivos, um número X só pode ser

divisor de Y se $X \leq Y$. Se conseguirmos garantir que **todos os elementos de c são estritamente maiores que todos os elementos de b **, a condição de divisor será impossível de ser satisfeita, pois um número maior nunca divide um número menor.

A estratégia mais simples é:

1. Encontrar o valor máximo presente no array a .
2. Colocar **todos** os elementos que possuem esse valor máximo no array c .
3. Colocar todo o restante dos elementos no array b .

Dessa forma, como b só contém números menores que o máximo, e c só contém o valor máximo, nenhum c_j poderá dividir um b_i . O único caso em que isso falha é se todos os números do array forem iguais (não sobraria nada para o array b), resultando em -1 .

Mental Chunking

- **Gatilho Visual:** Pedido para separar um array em dois grupos onde a divisibilidade entre eles é proibida.
- **Padrão Clássico:** Isolamento do Valor Extremo. Se $X > Y > 0$, então $X \nmid Y$. Agrupar os maiores valores em um lado e os menores no outro resolve problemas de divisibilidade/múltiplos.
- **Armadilha Comum:** Tentar separar por paridade (pares e ímpares) ou primos. Isso falha facilmente (ex: 2 divide 4). A separação por "maior valor" é a mais robusta.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     vector<int> a(n);
5     for (int i = 0; i < n; i++) cin >> a[i];
6 }
```

```

7  sort(a.begin(), a.end());
8
9  // Se o menor for igual ao maior, todos sao iguais
10 if (a[0] == a[n - 1]) {
11     cout << -1 << endl;
12     return;
13 }
14
15 vector<int> b, c;
16 int i = n - 1;
17 int max_val = a[n - 1];
18
19 // Pegamos todos os elementos que sao iguais ao maximo para o
20 // array c
21 while (i >= 0 && a[i] == max_val) {
22     c.push_back(a[i]);
23     i--;
24 }
25 // O que sobrou (que e menor que o maximo) vai para b
26 while (i >= 0) {
27     b.push_back(a[i]);
28     i--;
29 }
30
31 cout << b.size() << " " << c.size() << endl;
32 for (int x : b) cout << x << " ";
33 cout << endl;
34 for (int x : c) cout << x << " ";
35 cout << endl;
36 }

```

15 Botões (A. Buttons)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Anna e Katie foram parar em um laboratório secreto. No laboratório, existem $a + b + c$ botões.

- a botões só podem ser pressionados por Anna.
- b botões só podem ser pressionados por Katie.
- c botões podem ser pressionados por qualquer uma das duas.

Anna e Katie decidiram jogar um jogo, alternando turnos para pressionar esses botões. Anna faz o primeiro movimento. Cada botão pode ser pressionado no máximo uma vez. O jogo termina quando uma das garotas não consegue mais fazer um movimento no seu turno. Essa garota perde.

Determine quem vencerá se ambas jogarem de forma ideal.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 10^4$) — o número de casos de teste. Cada caso consiste em três inteiros a, b e c ($1 \leq a, b, c \leq 10^9$) — a quantidade de botões da Anna, da Katie e os botões compartilhados, respectivamente.

Saída: Para cada caso, imprima **First** se Anna vencer, ou **Second** se Katie vencer.

Exemplo

Input
5
1 1 1
9 3 3
1 2 3
6 6 9
2 2 8

Output
First
First
Second
First
Second

A Sacada do Problema

Em jogos de turnos onde ambos jogam de forma ideal, a prioridade máxima é sempre esgotar os recursos que o oponente também pode usar. Portanto, Anna e Katie vão pressionar todos os c botões compartilhados antes de tocarem em seus botões privados (a e b).

A vitória depende de quem tem mais fôlego após os botões compartilhados acabarem:

- **Se c é ímpar:** Anna (que começa) pressionará o último botão compartilhado. Isso significa que ela gastou uma rodada "extra" do pool comum. Agora é o turno de Katie, e Anna tem uma vantagem de +1 movimento. Anna vence se $a + 1 > b$, ou simplificando: $a \geq b$.
- **Se c é par:** Katie pressionará o último botão compartilhado. Ambos gastaram a mesma quantidade de turnos do pool comum. Agora é o

turno de Anna. Para vencer, Anna precisa ter estritamente mais botões privados que Katie: $a > b$.

Note que, como os números podem chegar a 10^9 , você nunca deve simular os turnos!

Mental Chunking

- **Gatilho Visual:** Jogo de turnos com recursos privados (A, B) e um recurso compartilhado (C).
- **Padrão Clássico:** Paridade de Recursos Compartilhados. Em jogos de "quem fica sem movimentos primeiro", analise quem leva a vantagem do último movimento no pool comum ($C\%2$).
- **Armadilha Comum:** Achar que precisa decidir qual botão Anna deve apertar primeiro. A estratégia ideal é sempre exaurir o recurso comum.

Solução (C++)

```
1 void solve() {
2     long long a, b, c;
3     cin >> a >> b >> c;
4
5     // Se C for ímpar, Anna ganha um "turno extra" de vantagem
6     // A disputa então é se  $a + (1 \text{ se } C \text{ ímpar}) > b$ 
7     if (c % 2 != 0) {
8         if (a >= b) cout << "First\n";
9         else cout << "Second\n";
10    }
11    // Se C for par, Anna precisa ter mais botoes proprios que
12    // Katie
13    else {
14        if (a > b) cout << "First\n";
15        else cout << "Second\n";
16    }
17 }
```

16 Colorindo o Array (A. Array Coloring)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Você recebe um array consistindo de n inteiros. Sua tarefa é determinar se é possível colorir todos os seus elementos em duas cores de tal forma que as somas dos elementos de ambas as cores tenham a mesma paridade (ambas pares ou ambas ímpares) e cada cor tenha pelo menos um elemento colorido.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 1000$) — o número de casos de teste. Cada caso começa com uma linha contendo um inteiro n ($2 \leq n \leq 50$) — o comprimento do array a . A próxima linha contém n inteiros $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 50$) — os elementos do array a .

Saída: Para cada caso de teste, imprima YES se for possível realizar a coloração, e NO caso contrário.

Exemplo

Input

```
7
8
1 2 4 3 2 3 5 4
2
4 7
3
3 9 8
2
1 7
5
5 4 3 2 1
4
4 3 4 5
2
50 48
```

Output

```
YES
NO
YES
YES
NO
YES
YES
```

A Sacada do Problema

Sejam S_1 e S_2 as somas dos elementos das duas cores. O problema pede que S_1 e S_2 tenham a mesma paridade. Pela álgebra da paridade, sabemos que:

- $Par + Par = Par$

- $mpar + mpar = Par$

Em ambos os casos permitidos pelo problema, a soma total do array ($S_{total} = S_1 + S_2$) resultará em um número **par**. Se S_1 e S_2 tivessem paridades diferentes ($Par + mpar$), a soma total seria ímpar.

Portanto, a condição necessária e suficiente é que a **soma total de todos os elementos do array seja par**. Como a paridade de uma soma depende apenas da quantidade de números ímpares: a soma total será par se, e somente se, o array contiver uma quantidade **par de números ímpares**.

Mental Chunking

- **Gatilho Visual:** Divisão de elementos em dois grupos com restrição de paridade na soma de cada grupo.
- **Padrão Clássico:** Álgebra de Paridade. A soma de duas partes tem a mesma paridade se, e somente se, o todo for par ($A \equiv B \pmod{2} \iff A + B \equiv 0 \pmod{2}$).
- **Armadilha Comum:** Tentar encontrar os subconjuntos usando força bruta ou bitmask. Basta somar tudo e checar o resto da divisão por 2.

```
15     cout << "YES\n";
16 } else {
17     cout << "NO\n";
18 }
19 }
```

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4
5     int soma_total = 0;
6     for (int i = 0; i < n; i++) {
7         int x;
8         cin >> x;
9         soma_total += x;
10    }
11
12    // Se a soma de todos os elementos for par,
13    // conseguimos dividir em duas partes de mesma paridade.
14    if (soma_total % 2 == 0) {
```

17 Desordenação (A. Desorting)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Chamamos um array a de comprimento n de **ordenado** se $a_1 \leq a_2 \leq \dots \leq a_{n-1} \leq a_n$.

Ntarsis possui um array a . Ele tem permissão para realizar um tipo de operação nele (zero ou mais vezes):

- Escolha um índice i ($1 \leq i \leq n - 1$).
- Adicione 1 a $a_1, a_2, \dots, a_i$.
- Subtraia 1 de $a_{i+1}, a_{i+2}, \dots, a_n$.

Determine o número mínimo de operações necessárias para tornar o array a **não ordenado**.

Entrada e Saída

Entrada: A primeira linha contém o número de casos de teste t ($1 \leq t \leq 100$). A primeira linha de cada caso contém um inteiro n ($2 \leq n \leq 500$) — o comprimento do array. A próxima linha contém n inteiros a_i ($1 \leq a_i \leq 10^9$) — os valores do array.

Saída: Para cada caso, imprima o número mínimo de operações para tornar o array não ordenado.

Exemplo

Input

```
4
2
1 1
4
1 8 10 13
3
1 3 2
3
1 9 14
```

Output

```
1
2
0
3
```

A Sacada do Problema

Para tornar um array "não ordenado", precisamos quebrar a condição $a_i \leq a_{i+1}$ para pelo menos um par de vizinhos. Ou seja, queremos que a_i se torne estritamente maior que a_{i+1} .

Observe o que a operação faz com a diferença $diff = a_{i+1} - a_i$:

1. a_i aumenta em 1.
2. a_{i+1} diminui em 1.

Isso significa que a cada operação, a distância entre eles **diminui em 2 unidades**.

Se a diferença inicial entre dois vizinhos é D , queremos saber quantas operações k são necessárias para que $D - 2k < 0$. A matemática nos diz que $2k > D$, logo $k = \lfloor \frac{D}{2} \rfloor + 1$.

A estratégia é: 1. Se o array já não estiver ordenado, o custo é 0. 2. Se estiver ordenado, calculamos qual par de vizinhos (a_i, a_{i+1}) tem a menor diferença entre eles. Esse é o nosso "gargalo" e onde será mais barato quebrar a ordenação.

Mental Chunking

- **Gatilho Visual:** Operação que altera prefixos e sufixos simultaneamente para quebrar uma propriedade global (ordenação).
- **Padrão Clássico:** Análise de Diferença entre Vizinhos (Delta). Em problemas de ordenação, o foco quase sempre deve estar na relação $a_{i+1} - a_i$.
- **Armadilha Comum:** Tentar simular a operação alterando todos os elementos do array. Como N é até 500 e os valores até 10^9 , simular causaria TLE. Foque apenas na mudança da diferença entre os pares.

```

17
18     if (ja_desordenado) {
19         cout << 0 << '\n';
20     } else {
21         // A cada passo a diferenca cai 2.
22         // Para ficar negativa: diff/2 + 1
23         cout << min_diff / 2 + 1 << '\n';
24     }
25 }

```

Solução (C++)

```

1 void solve() {
2     int n;
3     cin >> n;
4     vector<long long> a(n);
5     long long min_diff = 1e18;
6     bool ja_desordenado = false;
7
8     for (int i = 0; i < n; i++) {
9         cin >> a[i];
10        if (i > 0) {
11            // Se encontrarmos um par que quebra a ordem
12            if (a[i] < a[i-1]) ja_desordenado = true;
13            // Guardamos a menor diferenca entre vizinhos
14            min_diff = min(min_diff, a[i] - a[i-1]);
15        }
16    }

```

18 Inteiro Proibido (A. Forbidden Integer)

Tempo limite: 2 segundos
Memória limite: 256 megabytes

Enunciado

Você recebe um inteiro n que deseja obter. Você tem um suprimento ilimitado de todos os inteiros de 1 até k , exceto o inteiro x (o inteiro x não existe no seu estoque).

Você tem permissão para pegar uma quantidade arbitrária de cada um desses inteiros (possivelmente zero). Você consegue fazer com que a soma dos inteiros escolhidos seja igual a n ?

Se houver múltiplas respostas, imprima qualquer uma delas.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 100$) — o número de casos de teste. A única linha de cada caso contém três inteiros n, k e x ($1 \leq x \leq k \leq n \leq 100$).

Saída: Para cada caso, imprima YES ou NO. Se a resposta for YES, a segunda linha deve conter um inteiro m — a quantidade total de inteiros usados. A terceira linha deve conter os m inteiros cuja soma é n .

Exemplo

Input

```
5
10 3 2
5 2 1
4 2 1
7 7 3
6 1 1
```

Output

```
YES
6
3 1 1 1 1 3
NO
YES
2
2 2
YES
1
7
NO
```

A Sacada do Problema

O número ****1**** é o "átomo" da soma. Se ele estiver disponível, você sempre consegue formar qualquer número n simplesmente somando 1 várias vezes. O problema real só começa quando o $x = 1$.

Vamos analisar os cenários:

1. **Se $x \neq 1$:** O número 1 está disponível. Resposta sempre YES. Use n vezes o número 1.
2. **Se $x = 1$:** Você só pode usar números de 2 até k .

- Se $k = 1$: Não sobrou nenhum número para usar. NO.
- Se n for **Par**: Você pode usar apenas o número 2 várias vezes ($n/2$ vezes). Como $k \geq 2$, o 2 está disponível. YES.
- Se n for **Ímpar**: Para formar um número ímpar, você precisa de pelo menos um número ímpar disponível (como o 3).
 - Se $k < 3$ ou $n = 1$: Você não tem números ímpares ou o n é pequeno demais. NO.
 - Caso contrário: Use um número 3 e o restante em números 2. A soma será $3 + 2 \times (\frac{n-3}{2}) = n$. YES.

Mental Chunking

- **Gatilho Visual:** Formar uma soma N usando valores de um intervalo $[1, K]$ com uma exceção X .
- **Padrão Clássico:** Análise do Elemento Unitário. Se o 1 estiver disponível, o problema é trivial. Se não estiver, o foco muda para paridade (uso do 2 e do 3).
- **Armadilha Comum:** Tentar usar algoritmos de Programação Dinâmica (como o Problema do Troco). Embora funcione para $N \leq 100$, a análise matemática de casos é muito mais rápida para implementar em uma competição.

```

12     cout << "NO\n";
13   } else {
14     cout << "YES\n";
15     if (n % 2 == 0) {
16       cout << n / 2 << "\n";
17       for (int i = 0; i < n / 2; i++) cout << 2 << (i
== n/2 - 1 ? "" : " ");
18     } else {
19       // n e impar e k >= 3, usamos um 3 e o resto 2
20       cout << (n - 3) / 2 + 1 << "\n";
21       cout << 3 << " ";
22       for (int i = 0; i < (n - 3) / 2; i++) cout << 2
<< (i == (n-3)/2 - 1 ? "" : " ");
23     }
24     cout << "\n";
25   }
26 }
27 }

```

Solução (C++)

```

1 void solve() {
2     int n, k, x;
3     cin >> n >> k >> x;
4
5     if (x != 1) {
6         cout << "YES\n" << n << "\n";
7         for (int i = 0; i < n; i++) cout << 1 << (i == n - 1 ? ""
: " ");
8         cout << "\n";
9     } else {
10        // Se x == 1, precisamos usar 2 e 3
11        if (k == 1 || (n % 2 != 0 && k < 3)) {

```

19 Gafanhoto em uma Linha (A. Grasshopper on a Line)

Tempo limite: 2 segundos
Memória limite: 256 megabytes

Enunciado

Você recebe dois inteiros x e k . Um gafanhoto começa no ponto 0 em um eixo OX. Em um movimento, ele pode saltar uma distância inteira (para a esquerda ou para a direita) que **não seja divisível por k** .

Qual é o menor número de movimentos necessário para o gafanhoto chegar ao ponto x ? Quais são esses movimentos? Se houver múltiplas respostas, imprima qualquer uma delas.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 1000$) — o número de casos de teste. Cada caso contém dois inteiros x e k ($1 \leq x \leq 100; 2 \leq k \leq 100$) — o ponto de destino e a restrição dos saltos.

Saída: Para cada caso, imprima na primeira linha um inteiro n — o menor número de movimentos. Na segunda linha, imprima n inteiros que representem os saltos. A soma dos saltos deve ser exatamente x e nenhum deles pode ser divisível por k .

Exemplo

Input

```
3
10 2
10 3
3 4
```

Output

```
2
7 3
1
10
1
3
```

A Sacada do Problema

O objetivo é chegar em x com o mínimo de saltos. Vamos testar as possibilidades por ordem de simplicidade:

1. **É possível chegar com 1 salto?** Sim, se a distância total x não for divisível por k . Nesse caso, o gafanhoto simplesmente pula de 0 direto para x .
2. **E se x for divisível por k ?** Nesse caso, precisamos de pelo menos 2 saltos. A boa notícia é que **2 saltos sempre serão suficientes**. Podemos decompor x em dois saltos: $x - 1$ e 1.
 - O salto de tamanho 1 nunca é divisível por k (já que $k \geq 2$).
 - O salto de tamanho $x - 1$ também não será divisível por k se x for divisível por k . Pense na reta numérica: se x é um múltiplo de k , o número imediatamente anterior ($x - 1$) não pode ser múltiplo de k .

Assim, a resposta será sempre 1 ou 2 movimentos. Nunca mais que isso.

Mental Chunking

- **Gatilho Visual:** O problema pede o "número mínimo de passos" para atingir uma meta com uma "restrição de divisibilidade".
- **Padrão Clássico:** Decomposição em Casos Mínimos. Em problemas de movimento com restrições simples, a resposta costuma ser trivial (1 passo) ou uma pequena correção (2 passos usando $X - 1$ e 1).
- **Armadilha Comum:** Tentar usar Busca em Largura (BFS) ou Programação Dinâmica. Embora o x seja pequeno o suficiente para isso, é um desperdício de tempo e código para uma lógica que se resolve com um `if`.

Solução (C++)

```
1 void solve() {
2     int x, k;
3     cin >> x >> k;
4
5     // Caso 1: 0 proprio destino ja e um salto valido
6     if (x % k != 0) {
7         cout << 1 << '\n';
8         cout << x << '\n';
9     }
10    // Caso 2: Precisamos quebrar o salto em dois
11    else {
12        cout << 2 << '\n';
13        // Usamos a decomposicao universal (x-1) e 1
14        cout << x - 1 << " " << 1 << '\n';
15    }
16 }
```

20 Array Unitário (A. Unit Array)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Dado um array a de comprimento n , cujos elementos são iguais a -1 e 1 . Vamos chamar o array a de **bom** se as seguintes condições forem atendidas ao mesmo tempo:

- $a_1 + a_2 + \dots + a_n \geq 0$;
- $a_1 \cdot a_2 \cdot \dots \cdot a_n = 1$.

Em uma operação, você pode selecionar um elemento arbitrário do array a_i e mudar seu valor para o oposto (se for -1 , vira 1 ; se for 1 , vira -1).

Determine o número mínimo de operações necessárias para tornar o array a bom. É garantido que isso sempre é possível.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 500$) — o número de casos de teste. A primeira linha de cada caso contém um inteiro n ($1 \leq n \leq 100$) — o comprimento do array. A segunda linha contém n inteiros a_i ($a_i = \pm 1$).

Saída: Para cada caso de teste, imprima o número mínimo de operações para tornar o array bom.

Exemplo

Input

```
7
4
-1 -1 1 -1
5
-1 -1 -1 1 1
4
-1 1 -1 1
3
-1 -1 -1
5
1 1 1 1 1
1
-1
2
-1 -1
```

Output

```
1
1
0
3
0
1
2
```

A Sacada do Problema

Temos duas condições matemáticas para satisfazer com o menor número de trocas:

1. **A Soma (≥ 0):** Como o array só tem 1 e -1 , a soma será positiva ou zero

se a quantidade de números 1 for maior ou igual à quantidade de números -1 . 2. **O Produto ($= 1$):** O produto de vários números 1 e -1 só será 1 se a quantidade de números -1 for **par**.

A estratégia gulosa é: Primeiro, focamos na soma. Se houver mais -1 s do que 1s, transformamos os -1 s em 1s um por um até que a quantidade de -1 s seja menor ou igual à de 1s (ou seja, $cnt_{-1} \leq n/2$). Depois, olhamos para a quantidade resultante de -1 s. Se ela for ímpar, o produto será -1 . Para consertar isso sem quebrar a primeira regra, transformamos mais um -1 em 1. Agora a quantidade de -1 s será par e a soma continuará positiva.

Mental Chunking

- **Gatilho Visual:** Array com apenas dois valores possíveis (como -1 e 1) e restrições sobre a soma e o produto.
- **Padrão Clássico:** Balanceamento de Contagem. Em problemas de -1 e 1 , o produto é controlado pela paridade dos negativos e a soma pelo excesso de um sobre o outro.
- **Armadilha Comum:** Tentar mudar 1 para -1 . Como queremos aumentar a soma e manter o produto, quase sempre a operação correta é transformar -1 em 1.

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     int c1 = 0, cm1 = 0;
5     for (int i = 0; i < n; i++) {
6         int x;
7         cin >> x;
8         if (x == 1) c1++;
9         else cm1++;
10    }
11
12    int ops = 0;
13    // Regra 1: Soma >= 0 (mais 1s que -1s)
14    while (cm1 > c1) {
15        cm1--;
```

```

16         c1++;
17         ops++;
18     }
19
20     // Regra 2: Produto = 1 (quantidade de -1s deve ser par)
21     if (cm1 % 2 != 0) {
22         ops++;
23     }
24
25     cout << ops << '\n';
26 }

```

21 Permutações Gêmeas (A. Twin Permutations)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Você recebe uma permutação a de comprimento n .

Encontre qualquer permutação b de comprimento n tal que a sequência de somas $a_1 + b_1, a_2 + b_2, \dots, a_n + b_n$ seja não-decrescente. Ou seja, $a_1 + b_1 \leq a_2 + b_2 \leq \dots \leq a_n + b_n$.

Uma permutação de comprimento n é um array consistindo de n inteiros distintos de 1 a n em ordem arbitrária.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 2000$) — o número de casos de teste. A primeira linha de cada caso contém um inteiro n ($1 \leq n \leq 100$) — o comprimento das permutações. A segunda linha contém os n elementos da permutação a .

Saída: Para cada caso de teste, imprima qualquer permutação b que satisfaça a condição. É garantido que tal permutação sempre existe.

Exemplo

Input
5
5
1 2 4 5 3
2
1 2
1
1
3
3 2 1
4
1 4 3 2

Output
5 4 2 1 3
2 1
1
1 2 3
4 1 2 3

A Sacada do Problema

O problema pede que as somas sejam não-decrescentes. Em vez de tentarmos várias combinações, existe um truque matemático para tornar todas as somas exatamente ****iguais****.

Em uma permutação de 1 a n , o menor valor é 1 e o maior é n . Se quisermos que a soma de cada par (a_i, b_i) seja constante, podemos usar a fórmula:

$$a_i + b_i = n + 1$$

Dessa forma, $b_i = (n + 1) - a_i$.

Como a_i varia de 1 até n , o valor de b_i também variará de 1 até n . E como todos os a_i são distintos, todos os b_i resultantes também serão distintos, formando uma permutação válida. Se todas as somas são iguais a $n + 1$, então a condição $s_1 \leq s_2 \leq \dots \leq s_n$ é satisfeita (já que $n + 1 = n + 1$).

Mental Chunking

- **Gatilho Visual:** O problema pede para encontrar uma permutação que satisfaça uma desigualdade de somas $(a_i + b_i \leq a_{i+1} + b_{i+1})$.
- **Padrão Clássico:** Complemento de Permutação (Soma Constante). Para neutralizar o efeito de uma permutação, use seu "espelho" ou complemento: $b_i = (\min + \max) - a_i$.
- **Armadilha Comum:** Tentar ordenar o array de entrada ou usar uma estratégia gananciosa (Greedy) complexa. Tornar as somas constantes é a solução mais robusta e rápida.

Solução (C++)

```
1 void solve() {  
2     int n;  
3     cin >> n;  
4  
5     // Para cada elemento a[i], o b[i] correspondente  
6     // que mantém a soma constante e (n + 1) - a[i]  
7     for (int i = 0; i < n; i++) {  
8         int a;  
9         cin >> a;  
10        cout << (n + 1) - a << (i == n - 1 ? "" : " ");  
11    }  
12    cout << '\n';  
13 }
```


22 Espaço em Branco (B. Blank Space)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Você recebe um array binário a de n elementos. Um array binário é um array consistindo apenas de 0s e 1s.

Um **espaço em branco** é um segmento de elementos **consecutivos** consistindo apenas de 0s. Sua tarefa é encontrar o comprimento do maior espaço em branco no array.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 1000$) — o número de casos de teste. A primeira linha de cada caso contém um único inteiro n ($1 \leq n \leq 100$) — o comprimento do array. A segunda linha de cada caso contém n inteiros a_i separados por espaço ($0 \leq a_i \leq 1$) — os elementos do array.

Saída: Para cada caso de teste, imprima um único inteiro — o comprimento do maior espaço em branco.

Exemplo

Input

```
5
5
1 0 0 1 0
4
0 1 1 1
1
0
3
1 1 1
9
1 0 0 0 1 0 0 0 1
```

Output

```
2
1
1
0
3
```

A Sacada do Problema

Este é o clássico problema de encontrar o comprimento da maior sub-sequência contígua. A lógica consiste em percorrer o array e manter dois contadores:

1. **atual:** Conta quantos 0s estamos vendo em sequência agora.
2. **ans:** Guarda o maior valor que o contador **atual** já atingiu até o momento.

Ao encontrar um 0, incrementamos o contador **atual**. Ao encontrar um 1,

significa que o "espaço em branco" acabou: comparamos o tamanho desse espaço com o nosso recorde global (**ans**) e resetamos o contador **atual** para zero.

Mental Chunking

- **Gatilho Visual:** Pedido para encontrar o "maior segmento consecutivo" de um valor específico em um array.
- **Padrão Clássico:** Contador com Reset. Mantenha uma variável para o progresso atual e outra para o máximo global. O reset acontece sempre que o padrão é quebrado.
- **Armadilha Comum:** Esquecer de atualizar o máximo global (**ans**) no final do loop. Se o maior espaço em branco estiver no final do array, o 1 que dispararia o reset nunca será lido. Sempre faça um **max()** final ou atualize o recorde a cada incremento do contador.

```
22  
23     cout << ans << '\n';  
24 }
```

Solução (C++)

```
1 void solve() {  
2     int n;  
3     cin >> n;  
4  
5     int ans = 0;  
6     int atual = 0;  
7  
8     for (int i = 0; i < n; i++) {  
9         int x;  
10        cin >> x;  
11  
12        if (x == 0) {  
13            atual++;  
14            // Atualizamos o maximo global a cada passo  
15            // para nao perder o valor se o array terminar em 0  
16            ans = max(ans, atual);  
17        } else {  
18            // Se achou 1, o espaco em branco quebrou  
19            atual = 0;  
20        }  
21    }
```

23 Moedas (A. Coins)

Tempo limite: 2 segundos

Memória limite: 256 megabytes

Enunciado

Em Berland, existem dois tipos de moedas com valores de **2** e k burles.

Sua tarefa é determinar se é possível representar n burles usando essas moedas, ou seja, se existem inteiros não-negativos x e y tais que:

$$2 \cdot x + k \cdot y = n$$

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 10^4$) — o número de casos de teste. A única linha de cada caso contém dois inteiros n e k ($1 \leq n, k \leq 10^{18}$; $k \neq 2$).

Saída: Para cada caso, imprima YES se for possível representar n , ou NO caso contrário.

Exemplo

Input

```
4
5 3
6 1
7 4
8 8
```

Output

```
YES
YES
NO
YES
```

A Sacada do Problema

Queremos resolver a equação $2x + ky = n$. Como $2x$ é sempre **par**, a paridade de n depende inteiramente do termo ky .

Vamos analisar os dois cenários para n :

1. **Se n for Par:** Podemos simplesmente ignorar a moeda k (fazer $y = 0$) e usar apenas moedas de 2 ($x = n/2$). Como n é par, x será um inteiro.

Resposta: YES.

2. **Se n for Ímpar:** Para que a soma de um número par ($2x$) com ky resulte em um número ímpar, o termo ky **obrigatoriamente deve ser ímpar**.

- Para ky ser ímpar, k **deve ser ímpar** e y deve ser ímpar (podemos usar $y = 1$).
- Se k for par, ky será sempre par, e nunca conseguiremos chegar em um n ímpar.

Portanto, a lógica simplificada é: Se n é par, YES. Se n é ímpar, só é YES se k também for ímpar.

Mental Chunking

- **Gatilho Visual:** Equação do tipo $ax + by = c$ com constantes muito grandes (10^{18}).
- **Padrão Clássico:** Análise de Paridade. Em somas envolvendo o número 2, a paridade do resultado é ditada pelo outro termo. Se o objetivo é ímpar, você *precisa* de um componente ímpar disponível.
- **Armadilha Comum:** Tentar usar o Algoritmo de Euclides Estendido ou DP. Para este problema específico (onde um dos coeficientes é 2), a paridade simples resolve tudo sem precisar de álgebra pesada.

Solução (C++)

```
1 void solve() {
2     long long n, k;
3     cin >> n >> k;
4
5     // Se n e par, basta usar moedas de 2.
6     // Se n e impar, precisamos que k seja impar para
7     // "subtrair" 1 de n e torna-lo par.
8     if (n % 2 == 0) {
9         cout << "YES\n";
10    } else {
11        // Se n e impar e k e impar, entao (n-k) e par.
12        // Se n >= k, podemos usar uma moeda k e o resto de 2.
13        if (k % 2 != 0 && n >= k) {
14            cout << "YES\n";
15        } else {
16            cout << "NO\n";
17        }
18    }
19 }
```

24 Mestre da Caminhada (A. Walking Master)

Tempo limite: 1 segundo

Memória limite: 1024 megabytes

Enunciado

YunQian está em um plano infinito com um sistema de coordenadas cartesianas. Em um movimento, ela pode se mover para o ponto adjacente diagonalmente no topo direito ou para o ponto adjacente à esquerda.

Ou seja, se ela estiver no ponto (x, y) , ela pode se mover para $(x + 1, y + 1)$ ou para $(x - 1, y)$.

YunQian começa no ponto (a, b) e quer se mover para o ponto (c, d) . Encontre o número mínimo de movimentos que ela precisa fazer ou declare que é impossível.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 10^4$) — o número de casos de teste. Cada caso consiste em quatro inteiros a, b, c, d ($-10^8 \leq a, b, c, d \leq 10^8$).

Saída: Para cada caso, imprima o número mínimo de movimentos. Se for impossível, imprima -1 .

Exemplo

Input

```
6
-1 0 -1 2
0 0 4 5
-2 -1 1 1
-3 2 -3 2
2 -1 -1 -1
1 1 0 2
```

Output

```
4
6
-1
0
3
3
```

A Sacada do Problema

Para resolver este problema, precisamos analisar como cada movimento afeta as coordenadas x e y separadamente:

- **Movimento 1** ($x + 1, y + 1$): É a única forma de aumentar y .
- **Movimento 2** ($x - 1, y$): Altera apenas x para a esquerda.

Isso nos leva a duas conclusões imediatas: 1. **Impossibilidade por Altura:** Como nenhum movimento diminui y , se $d < b$, é impossível chegar ao destino. 2. **Estratégia Obrigatória:** Para subir de b até d , YunQian *precisa* usar o Movimento 1 exatamente $(d - b)$ vezes.

Após usar esses $(d - b)$ movimentos, a nova coordenada x será $a + (d - b)$. Agora, ela está na altura correta d , mas sua posição horizontal é $a + d - b$. A

única forma de ajustar o x agora é usando o Movimento 2 (esquerda). Isso só é possível se o destino c for menor ou igual à posição atual. Se $c > a + d - b$, é **impossível**.

O total de movimentos será: $(d - b)$ [para subir] + $(a + d - b - c)$ [para ajustar o x à esquerda].

Mental Chunking

- **Gatilho Visual:** Movimentos restritos em um plano 2D onde uma das coordenadas só pode aumentar (movimento unidirecional em um eixo).
- **Padrão Clássico:** Decomposição de Movimentos. Resolva primeiro o eixo restrito (neste caso, o y) para descobrir a posição intermediária e depois calcule o ajuste no eixo livre (x).
- **Armadilha Comum:** Tentar usar BFS ou algoritmos de busca. As coordenadas chegam a 10^8 , o que exige uma solução matemática de tempo constante $O(1)$.

Solução (C++)

```
1 void solve() {
2     long long a, b, c, d;
3     cin >> a >> b >> c >> d;
4
5     // d < b: impossível descer
6     // c > a + (d - b): impossível ir para a direita alem do que
7     // a diagonal permite
8     if (d < b || c > a + d - b) {
9         cout << -1 << '\n';
10    } else {
11        // (d - b) movimentos para cima/diagonal
12        // (a + d - b - c) movimentos para a esquerda
13        long long ans = (d - b) + (a + d - b - c);
14        cout << ans << '\n';
15    }
```

25 Precisamos do Zero (A. We Need the Zero)

Tempo limite: 1 segundo
Memória limite: 256 megabytes

Enunciado

Existe um array a consistindo de inteiros não-negativos. Você pode escolher um inteiro x e denotar $b_i = a_i \oplus x$ para todo $1 \leq i \leq n$, onde $\oplus$ denota a operação **bitwise XOR**.

É possível escolher um número x tal que o valor da expressão $b_1 \oplus b_2 \oplus \dots \oplus b_n$ seja igual a 0?

Pode-se mostrar que, se um número x válido existir, então também existe um x tal que $(0 \leq x < 2^8)$.

Entrada e Saída

Entrada: A primeira linha contém o número de casos de teste t ($1 \leq t \leq 1000$). A primeira linha de cada caso contém um inteiro n ($1 \leq n \leq 10^3$) — o comprimento do array a . A segunda linha contém n inteiros a_i ($0 \leq a_i < 2^8$).

Saída: Para cada caso de teste, imprima o inteiro x ($0 \leq x < 2^8$) se ele existir, ou -1 caso contrário.

Exemplo

Input
5
3
1 2 5
3
1 2 3
4
0 1 2 3
4
1 2 2 3
1
1

Output
6
0
3
-1
1

A Sacada do Problema

A operação XOR ($\oplus$) possui propriedades muito úteis: ela é comutativa, associativa e $x \oplus x = 0$. A expressão que queremos zerar é: $(a_1 \oplus x) \oplus (a_2 \oplus x) \oplus \dots \oplus (a_n \oplus x) = 0$

Podemos reordenar isso para: $(a_1 \oplus a_2 \oplus \dots \oplus a_n) \oplus \underbrace{(x \oplus x \oplus \dots \oplus x)}_{n \text{ vezes}} = 0$

Seja S o resultado do XOR sum original do array a . Temos dois casos baseados no tamanho n :

1. **Se n for par:** O número x aparece um número par de vezes. Como $x \oplus x = 0$, todos os x se anulam. A expressão final é apenas S . Se $S = 0$, qualquer x serve (podemos responder 0). Se $S \neq 0$, é impossível zerar a expressão, pois o x não tem poder de alterar o resultado.
2. **Se n for ímpar:** O número x aparece um número ímpar de vezes. Os pares se anulam e sobra exatamente um x . A expressão final é $S \oplus x$. Para que $S \oplus x = 0$, basta escolhermos $x = S$.

Mental Chunking

- **Gatilho Visual:** Operações bitwise aplicadas a todos os elementos de um array seguidas de uma redução (XOR sum).
- **Padrão Clássico:** Paridade de Contribuição. Em problemas de XOR, verifique se a variável escolhida aparece um número par ou ímpar de vezes no resultado final. Se for par, ela é irrelevante; se for ímpar, ela pode neutralizar o valor acumulado.
- **Armadilha Comum:** Tentar fazer um loop de 0 a 255 para testar todos os x . Embora funcione neste problema específico (devido ao limite pequeno de 2^8), entender a lógica da paridade é o que permite resolver variações onde x pode ser muito maior.

```
15 // Se n e par, o xor_sum original precisa ser 0
16 if (xor_sum == 0) cout << 0 << '\n';
17 else cout << -1 << '\n';
18 }
19 }
```

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     int xor_sum = 0;
5     for (int i = 0; i < n; i++) {
6         int a;
7         cin >> a;
8         xor_sum ^= a;
9     }
10
11     if (n % 2 != 0) {
12         // Se n e impar, x = xor_sum resolve
13         cout << xor_sum << '\n';
14     } else {
```

26 Adicionar no Início e no Fim (C. Prepend and Append)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Timur inicialmente tinha uma string binária s (possivelmente de comprimento 0). Ele realizou a seguinte operação várias vezes:

- Adicionar **0** em uma extremidade da string e **1** na outra extremidade. Por exemplo, começando da string 1011, ele poderia obter 010111 ou 110110.

Você recebe a string final de Timur. Qual é o comprimento da **menor** string possível com a qual ele poderia ter começado?

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 100$) — o número de casos de teste. A primeira linha de cada caso contém um inteiro n ($1 \leq n \leq 2000$) — o comprimento da string final. A segunda linha contém a string s de comprimento n .

Saída: Para cada caso de teste, imprima o menor comprimento possível da string original. Se a original puder ser vazia, imprima 0.

Exemplo

Input

```
9
3
100
4
0111
5
10101
6
101010
7
1010110
1
1
2
10
2
11
10
1011011010
```

Output

```
1
2
5
0
3
1
0
2
4
```


A Sacada do Problema

A operação de Timur sempre adiciona caracteres ****diferentes**** nas duas pontas (um '0' e um '1'). Para encontrar a string original mais curta, devemos "desfazer" essa operação o máximo de vezes possível.

Como desfazer? Olhamos para o primeiro e o último caractere da string atual:

- Se eles forem **diferentes** (um é '0' e o outro é '1'), então eles podem ter sido adicionados em uma operação. Nós "removemos" esses dois caracteres e diminuimos o tamanho em 2.
- Se eles forem **iguais** (ambos '0' ou ambos '1'), a operação de Timur **não poderia** ter produzido esse par. Isso significa que chegamos na string original e não podemos mais reduzir.

Usamos dois ponteiros: `l` começando no início e `r` no fim, movendo-os para o centro enquanto os caracteres forem opostos.

Mental Chunking

- **Gatilho Visual:** Operações que modificam simultaneamente as duas extremidades de uma sequência.
- **Padrão Clássico:** Dois Ponteiros Invertidos (Outside-In). Comece nos extremos e processe em direção ao centro enquanto uma condição de simetria ou oposição for mantida.
- **Armadilha Comum:** Esquecer de parar quando os ponteiros se cruzam ($l > r$) ou quando a string tem tamanho 1.

Solução (C++)

```
1 void solve() {  
2     int n;  
3     cin >> n;  
4     string s;  
5     cin >> s;
```

```
6  
7     int l = 0, r = n - 1;  
8     int tamanho_atual = n;  
9  
10    // Enquanto o caractere da esquerda for diferente do da  
11    // direita  
12    // e os ponteiros não se cruzarem  
13    while (l < r && s[l] != s[r]) {  
14        l++;  
15        r--;  
16        tamanho_atual -= 2;  
17    }  
18  
19    cout << tamanho_atual << '\n';  
20 }
```

27 O Array de Serval e Mocha (A. Serval and Mocha's Array)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Mocha gosta de arrays, e Serval deu a ela um array de inteiros positivos. Mocha define as seguintes propriedades:

- Um array é **bom** se o MDC de todos os seus elementos não for maior que o seu comprimento.
- Um array de pelo menos 2 elementos é **bonito** se todos os seus prefixos de comprimento ≥ 2 forem **bons**.

Dada um array a de n inteiros, determine se é possível reordenar seus elementos para torná-lo **bonito**.

Entrada e Saída

Entrada: A primeira linha contém t ($1 \leq t \leq 500$) — o número de casos de teste. Cada caso começa com n ($2 \leq n \leq 100$) — o comprimento do array. A segunda linha contém os n inteiros a_i ($1 \leq a_i \leq 10^6$).

Saída: Para cada caso, imprima **Yes** se for possível tornar o array bonito através de reordenação, e **No** caso contrário.

Exemplo

Input

```
6
2
3 6
3
1 2 4
3
3 6 1
3
15 35 21
4
35 10 35 14
5
1261 227821 143 4171 1941
```

Output

```
No
Yes
Yes
No
Yes
Yes
```

A Sacada do Problema

A chave aqui é a propriedade de monotonicidade do MDC. O MDC de um conjunto de números **nunca aumenta** quando você adiciona mais números a esse conjunto.

Se conseguirmos que o **primeiro prefixo** (de comprimento 2) seja bom, ou seja, $\gcd(a_1, a_2) \leq 2$, então todos os prefixos seguintes também serão bons. Por quê?

1. Se $\gcd(a_1, a_2) \leq 2$, o prefixo de tamanho 2 é bom.
2. Para qualquer prefixo de tamanho $L > 2$, o MDC desse prefixo será $\leq \gcd(a_1, a_2)$.
3. Como $\gcd(a_1, a_2) \leq 2$ e o comprimento $L > 2$, então o MDC certamente será $\leq L$.

Portanto, o problema se resume a: existe **qualquer par** de elementos no array original cujo MDC seja menor ou igual a 2? Se sim, podemos colocar esses dois no início e o array será bonito.

Mental Chunking

- **Gatilho Visual:** Uma condição de prefixo que envolve MDC e o comprimento do prefixo (L).
- **Padrão Clássico:** MDC Monotônico. O MDC de um prefixo só diminui ou estabiliza conforme o prefixo cresce. Se a condição é satisfeita para o menor caso ($L = 2$), ela tende a se manter para os maiores.
- **Armadilha Comum:** Tentar ordenar o array de forma crescente ou decrescente. A ordem não importa tanto quanto a existência de *pelo menos um* par "chave" com $\text{MDC} \leq 2$.

Solução (C++)

```

1 // Funcao gcd ja existe no C++17 como std::gcd
2 void solve() {
3     int n;
4     cin >> n;
5     vector<int> a(n);
6     for (int i = 0; i < n; i++) cin >> a[i];
7
8     bool possivel = false;
9     // Testamos todos os pares possiveis (N e pequeno, ate 100)
10    for (int i = 0; i < n; i++) {
11        for (int j = i + 1; j < n; j++) {
12            if (__gcd(a[i], a[j]) <= 2) {

```

```

13                possivel = true;
14                break;
15            }
16        }
17        if (possivel) break;
18    }
19
20    if (possivel) cout << "Yes\n";
21    else cout << "No\n";
22 }

```

28 Um e Dois (A. One and Two)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Você recebe uma sequência $a_1, a_2, \dots, a_n$. Cada elemento de a é **1** ou **2**.

Descubra se existe um inteiro k ($1 \leq k \leq n - 1$) tal que o produto dos primeiros k elementos seja igual ao produto dos elementos restantes:

$$a_1 \cdot a_2 \cdot \dots \cdot a_k = a_{k+1} \cdot a_{k+2} \cdot \dots \cdot a_n$$

Se existirem múltiplos valores de k que satisfaçam a condição, imprima o **menor** deles. Se não existir nenhum, imprima -1 .

Entrada e Saída

Entrada: A primeira linha contém o número de casos de teste t ($1 \leq t \leq 100$). A primeira linha de cada caso contém o inteiro n ($2 \leq n \leq 1000$). A segunda linha contém n inteiros a_i ($1 \leq a_i \leq 2$).

Saída: Para cada caso, imprima o menor k possível ou -1 .

Exemplo

Input

```
3
6
2 2 1 2 1 2
3
1 2 1
4
1 1 1 1
```

Output

```
2
-1
1
```

A Sacada do Problema

A primeira coisa a notar é que o número **1** é o elemento neutro da multiplicação. Ele não altera o produto de forma alguma. O que realmente define o produto de um segmento é a **quantidade de números 2** que ele contém.

Se um segmento tem três números 2, o seu produto é $2^3 = 8$. Se outro segmento tem três números 2, o seu produto também é 8, não importa quantos números 1 estejam espalhados ali.

Logo, para que os dois produtos sejam iguais, ambos os lados do corte k devem ter exatamente a **mesma quantidade de números 2**.

1. Conte o total de números 2 no array (vamos chamar de **total**).
2. Se **total** for um número ímpar, é impossível dividi-lo igualmente em dois grupos. Resposta: -1 .
3. Se **total** for zero, qualquer corte serve (pois $1 = 1$). O menor k é 1.

4. Se `total` for par, percorra o array novamente e pare assim que encontrar a $(\text{total}/2)$ -ésima ocorrência do número 2. Esse índice será o seu k .

Mental Chunking

- **Gatilho Visual:** Comparação de produtos em arrays com valores muito restritos (como $\{1, 2\}$ ou $\{0, 1\}$).
- **Padrão Clássico:** Transformação de Produto em Contagem. Quando os valores são potências de uma mesma base (ou neutros), comparar o produto é o mesmo que comparar a frequência dos expoentes.
- **Armadilha Comum:** Tentar calcular o produto real. Como $n = 1000$, o produto pode chegar a 2^{1000} , o que estoura qualquer variável numérica padrão (`long long` só vai até $\sim 2^{63}$).

```
23 int current2 = 0;
24 for (int i = 0; i < n; i++) {
25     if (a[i] == 2) current2++;
26     // Quando chegarmos na metade dos 2s, achamos o corte k
27     if (current2 == total2 / 2) {
28         cout << i + 1 << '\n'; // +1 porque k e 1-indexed
29         return;
30     }
31 }
32 }
```

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     vector<int> a(n);
5     int total2 = 0;
6     for (int i = 0; i < n; i++) {
7         cin >> a[i];
8         if (a[i] == 2) total2++;
9     }
10
11     // Se o total de 2s for impar, nao tem como dividir igual
12     if (total2 % 2 != 0) {
13         cout << -1 << '\n';
14         return;
15     }
16
17     // Se nao houver nenhum 2, o produto e sempre 1, menor k e 1
18     if (total2 == 0) {
19         cout << 1 << '\n';
20         return;
21     }
22 }
```

29 Torne-o Bonito (A. Make it Beautiful)

Tempo limite: 3 segundos
Memória limite: 512 megabytes

Enunciado

Um array a é chamado de **feio** se ele contém pelo menos um elemento que é igual à **soma de todos os elementos antes dele**. Se o array não é feio, ele é **bonito**.

Por exemplo:

- $[6, 3, 9, 6]$ é feio: o elemento 9 é igual a $6 + 3$;
- $[5, 5, 7]$ é feio: o elemento 5 (o segundo) é igual ao anterior 5;
- $[8, 4, 10, 14]$ é bonito: $8 \neq 0, 4 \neq 8, 10 \neq 8 + 4, 14 \neq 8 + 4 + 10$.

Você recebe um array a já ordenado de forma não-decrescente ($a_1 \leq a_2 \leq \dots \leq a_n$). Você deve reordenar os elementos de a de modo que o array resultante seja bonito. Se for impossível, informe.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 2000$) — o número de casos de teste. Cada caso consiste em duas linhas: a primeira com n ($2 \leq n \leq 50$) e a segunda com n inteiros a_i ($1 \leq a_i \leq 100$). *Nota: o array de entrada já vem ordenado.*

Saída: Se for impossível, imprima NO. Caso contrário, imprima YES e o array reordenado.

Exemplo

Input

```
4
4
3 3 6 6
2
10 10
5
1 2 3 4 5
3
1 4 4
```

Output

```
YES
3 6 3 6
NO
YES
5 1 2 3 4
YES
1 4 4
```

A Sacada do Problema

Como o array de entrada está ordenado ($a_1 \leq a_2 \leq \dots \leq a_n$), o maior risco de ele ser "feio" é quando temos elementos repetidos ou quando a soma cresce linearmente.

O único caso onde é **impossível** tornar o array bonito é quando **todos os elementos são iguais**. Se $a_1 = a_n$, qualquer ordem que você escolher fará com que o segundo elemento seja igual ao primeiro ($a_2 = a_1$), quebrando a regra.

Para todos os outros casos, a estratégia mais simples é quebrar a "escada" de

somas. Se colocarmos o **maior elemento primeiro** e o **menor elemento logo em seguida**, a soma acumulada após o primeiro elemento já será o valor máximo do array. Como todos os elementos restantes são menores ou iguais a esse máximo, nenhum deles conseguirá atingir o valor da nova soma acumulada (que só tende a aumentar).

Mental Chunking

- **Gatilho Visual:** Condição baseada na "soma de todos os elementos anteriores" (*PrefixSum*).
- **Padrão Clássico:** Inversão de Extremos. Para evitar que um elemento iguale a soma acumulada, faça a soma saltar para um valor muito alto logo de cara (colocando o máximo no início).
- **Armadilha Comum:** Tentar fazer permutações aleatórias ou usar `std::next_permutation`. Com $N = 50$, o número de permutações é astronômico. A solução gulosa de mover o maior para a frente é $O(1)$ ou $O(N)$.

```
20     }
21 }
```

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     vector<int> a(n);
5     for (int i = 0; i < n; i++) cin >> a[i];
6
7     // Como ja esta ordenado, se o primeiro for igual ao ultimo,
8     // todos sao iguais.
9     if (a[0] == a[n - 1]) {
10         cout << "NO\n";
11     } else {
12         cout << "YES\n";
13         // Estrategia: coloca o maior primeiro, depois o menor,
14         // e segue a ordem original.
15         cout << a[n - 1] << " " << a[0] << " ";
16         for (int i = 1; i < n - 1; i++) {
17             cout << a[i] << (i == n - 2 ? " " : " ");
18         }
19         cout << "\n";
20     }
21 }
```

30 Todo Mundo Gosta de Arrays Bons (A. Everybody Likes Good Arrays!)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Um array a é **bom** se todos os pares de elementos adjacentes, a_i e a_{i+1} , possuem **paridade diferente** (um é par e o outro é ímpar).

Você pode realizar a seguinte operação: selecionar dois elementos adjacentes com a **mesma paridade**, removê-los e inserir o produto deles na mesma posição.

Encontre o número mínimo de operações para tornar o array bom.

Entrada e Saída

Entrada: A primeira linha contém t ($1 \leq t \leq 500$) — o número de casos de teste. Cada caso começa com n ($1 \leq n \leq 100$) — o comprimento do array. A segunda linha contém os n inteiros a_i ($1 \leq a_i \leq 10^9$).

Saída: Para cada caso, imprima o número mínimo de operações necessárias.

Exemplo

Input

```
3
5
1 7 11 2 13
4
1 2 3 4
6
1 1 1 2 2 3
```

Output

```
2
0
3
```

A Sacada do Problema

A chave aqui é entender como a multiplicação afeta a paridade:

- $\text{Par} \times \text{Par} = \text{Par}$
- $\text{Ímpar} \times \text{Ímpar} = \text{Ímpar}$

Isso significa que fundir dois números de mesma paridade resulta em um novo número que mantém essa mesma paridade. Na prática, a operação serve apenas para "encolher" blocos contíguos de números pares ou blocos de números ímpares.

Se você tem um bloco de k elementos seguidos com a mesma paridade, você precisará de exatamente $k - 1$ operações para transformá-los em um único elemento. O problema se resume a contar quantas vezes dois números vizinhos têm a mesma paridade e somar essas ocorrências.

Mental Chunking

- **Gatilho Visual:** Condição de adjacência baseada em paridade alternada.
- **Padrão Clássico:** Contagem de Consecutivos (Same-Parity Blocks). O custo de reduzir um bloco de tamanho K para tamanho 1 é sempre $K - 1$.
- **Armadilha Comum:** Tentar calcular o produto real dos números. Como os valores chegam a 10^9 , o produto de apenas três números já estouraria um `long long`. Use apenas o operador de resto (`% 2`).

Solução (C++)

```
1 void solve() {
2     int n;
3     cin >> n;
4     vector<int> a(n);
5     for (int i = 0; i < n; i++) cin >> a[i];
6
7     int ops = 0;
8     for (int i = 0; i < n - 1; i++) {
9         // Se o vizinho tem a mesma paridade, 1 operacao e
          necessaria
10         if (a[i] % 2 == a[i+1] % 2) {
11             ops++;
12         }
13     }
14     cout << ops << '\n';
15 }
```

31 Extremamente Redondo (A. Extremely Round)

Tempo limite: 3 segundos

Memória limite: 512 megabytes

Enunciado

Chamamos um inteiro positivo de **extremamente redondo** se ele possui apenas um dígito diferente de zero. Por exemplo, 5000, 4, 1, 10, 200 são inteiros extremamente redondos; 42, 13, 666, 77, 101 não são.

Você recebe um inteiro n . Você deve calcular o número de inteiros extremamente redondos x tais que $1 \leq x \leq n$.

Entrada e Saída

Entrada: A primeira linha contém um inteiro t ($1 \leq t \leq 10^4$) — o número de casos de teste. As próximas t linhas contém um inteiro n ($1 \leq n \leq 999.999$) — a descrição de cada caso.

Saída: Para cada caso de teste, imprima um inteiro representando a quantidade de números extremamente redondos até n .

Exemplo

Input
5
9
42
13
100
111

Output
9
13
10
19
19

A Sacada do Problema

Números extremamente redondos são números na forma $d \cdot 10^p$, onde $d \in \{1, 2, \dots, 9\}$ e $p \geq 0$. Isso significa que, para cada ordem de grandeza (número de dígitos), existem exatamente **9 números extremamente redondos**.

Para um número n qualquer:

1. Contamos todos os números redondos que possuem **menos dígitos** que n . Se n tem k dígitos, existem $(k - 1) \times 9$ desses números.
2. Contamos os números redondos que possuem a **mesma quantidade de dígitos** que n . Estes serão os números $10^{k-1}, 2 \cdot 10^{k-1}, \dots, d \cdot 10^{k-1}$, onde d é o **primeiro dígito** de n .

A fórmula final é simplesmente: $(k - 1) \times 9 + \text{primeiro_digito}$.

Mental Chunking

- **Gatilho Visual:** Contagem de números com propriedades digitais específicas (como "apenas um dígito não-zero").
- **Padrão Clássico:** Contagem por Magnitude. Em vez de iterar de 1 a N , conte quantos candidatos existem em cada "casa decimal" ($1 - 9$, $10 - 90$, $100 - 900$, etc.).
- **Armadilha Comum:** Tentar converter o número para string e iterar desnecessariamente. Embora funcione para $N \leq 10^6$, a lógica matemática de ordens de grandeza é mais escalável.

Solução (C++)

```
1 void solve() {
2     string s;
3     cin >> s;
4
5     // k e o numero de digitos do numero n
6     int k = s.size();
7
8     // O primeiro digito de n nos diz quantos redondos
9     // existem na maior ordem de grandeza.
10    int primeiro_digito = s[0] - '0';
11
12    // Cada ordem de grandeza abaixo de k contribui com 9 numeros
13    .
14    int ans = (k - 1) * 9 + primeiro_digito;
15
16    cout << ans << '\n';
17 }
```

32 Duas Permutações (A. Two Permutations)

Tempo limite: 1 segundo

Memória limite: 256 megabytes

Enunciado

Você recebe três inteiros n, a e b . Determine se existem duas permutações p e q de comprimento n para as quais as seguintes condições sejam verdadeiras:

- O comprimento do **prefixo comum mais longo** de p e q é a .
- O comprimento do **sufixo comum mais longo** de p e q é b .

Uma permutação de comprimento n é um array contendo cada inteiro de 1 a n exatamente uma vez.

Entrada e Saída

Entrada: A primeira linha contém um único inteiro t ($1 \leq t \leq 10^4$) — o número de casos de teste. A única linha de cada caso de teste contém três inteiros n, a e b ($1 \leq a, b \leq n \leq 100$).

Saída: Para cada caso de teste, se tal par de permutações existir, imprima **Yes**; caso contrário, imprima **No**.

Exemplo

Input

```
4
1 1 1
2 1 2
3 1 1
4 1 1
```

Output

```
Yes
No
No
Yes
```

A Sacada do Problema

Para resolver este problema, precisamos pensar no espaço que sobra no "meio" do array entre o prefixo e o sufixo.

1. **O Caso da Identidade:** Se $a = n$ e $b = n$, as duas permutações são idênticas. Isso só é possível se o comprimento total n for igual a a e b . Se $n = a = b$, a resposta é **Yes**.
2. **O Conflito de Extremidades:** Se as permutações não forem idênticas, o prefixo comum (a) e o sufixo comum (b) não podem se sobrepor de forma a "forçar" os elementos restantes a serem iguais.
3. **A Regra do Espaço Mínimo:** Para garantir que o prefixo termine **exatamente** em a e o sufixo comece **exatamente** em $n - b + 1$, precisamos de pelo menos **dois elementos** de folga no meio.
 - Por que dois? Se sobrar apenas um elemento no meio, como os elementos do prefixo e do sufixo já estão fixos, esse último elemento

também teria que ser igual em ambas as permutações, o que aumentaria o comprimento do prefixo ou do sufixo.

- Portanto, a condição para não-identidade é: $a + b \leq n - 2$.

Resumindo: a resposta é **Yes** se $(n = a = b)$ OU se $(a + b \leq n - 2)$.

Mental Chunking

- **Gatilho Visual:** Problemas de "Prefix/Suffix" comum em permutações com comprimentos fixos.
- **Padrão Clássico:** O "Buraco no Meio". Para que duas restrições de igualdade nas pontas não se fundam em uma igualdade total, você precisa de espaço suficiente entre elas para criar uma diferença (≥ 2 elementos).
- **Armadilha Comum:** Achar que $a + b < n$ é suficiente. Se $a + b = n - 1$, o elemento que sobra é forçado a ser o mesmo (pois é o único que resta da permutação), invalidando o comprimento "exato" de a ou b .

Solução (C++)

```
1 void solve() {
2     int n, a, b;
3     cin >> n >> a >> b;
4
5     // Caso 1: Sao a mesma permutacao
6     if (n == a && a == b) {
7         cout << "Yes\n";
8     }
9     // Caso 2: Ha espaco suficiente para criar uma diferenca no
10    meio
11    else if (a + b <= n - 2) {
12        cout << "Yes\n";
13    }
14    // Caso 3: As restricoes se chocam ou forcaram a identidade
15    else {
16        cout << "No\n";
17    }
18 }
```

“A persistência é o caminho do êxito.”

Agora que revisamos os problemas de rating 800, o próximo desafio nos espera.

Bons treinos e rumo ao próximo nível!
